

SOLIDWORKS API (.NET VBA C#) — Examples (41/50):

Inspect_Facets_Example_CSharp-

Measure_Selected_Entities_Example_VB

SOLIDWORKS API Documentation (converted from CHM)

Inspect Facets of Graphics and Mesh Bodies Example (C#)

This example shows how to inspect facets, facet edges, and facet vertexes after importing graphics and mesh bodies.

```
// =====

// Preconditions:

// 1. Copy Public_Documents\SOLIDWORKS\SOLIDWORKS 2022\samples\tutorial\api\Part1.PLY to c:\temp.

// 2. Open the Immediate window.

// Postconditions:

// 1. Press F5 five times, inspecting the Immediate window after each press.

// 2. The part is imported first as a graphics body and then as a mesh body.

// 3. The macro tests facets, facet edges, and facet vertexes after each import.

// =====

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Diagnostics;

namespace Macro1_CSharp
{
    public partial class SolidWorksMacro
    {
        private ModelDoc2 swModelDoc;

        public void Main()
        {
            swModelDoc = null;
            TestGraphicsBody();
            TestBodies();
            swModelDoc = null;
            swApp.CloseDoc("Part1");
            TestMeshBody();
            TestBodies();
            swModelDoc = null;
            swApp.CloseDoc("Part1");
        }
    }
}
```

```

    }

    public void TestGraphicsBody()
    {
        Debug.Print("Importing Part1.PLY as a graphics body...");
        Debug.Print("");
        bool boolstatus;
        boolstatus =
swApp.SetUserPreferenceIntegerValue((int)swUserPreferenceIntegerValue_e.swImportStlVrmlModelType,
(int)swImportStlVrmlModelType_e.swImportStlVrmlModelType_Graphics);
        boolstatus =
swApp.SetUserPreferenceIntegerValue((int)swUserPreferenceIntegerValue_e.swImportStlVrmlModelType,
0);
        boolstatus = swApp.LoadFile2("c:\temp\Part1.PLY", "r");
        swModelDoc = (ModelDoc2)swApp.ActiveDoc;
        SelectGraphicsBody();
    }

    public void TestMeshBody()
    {
        Debug.Print("Importing Part1.PLY as a mesh body...");
        Debug.Print("");
        bool boolstatus;
        boolstatus =
swApp.SetUserPreferenceIntegerValue((int)swUserPreferenceIntegerValue_e.swImportStlVrmlModelType,
(int)swImportStlVrmlModelType_e.swImportStlVrmlModelType_Surface);
        boolstatus =
swApp.SetUserPreferenceIntegerValue((int)swUserPreferenceIntegerValue_e.swImportStlVrmlModelType,
1);
        swApp.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swVrmlStlImportAsPSMesh,
true);
        boolstatus = swApp.LoadFile2("c:\temp\Part1.PLY", "r");
        swModelDoc = (ModelDoc2)swApp.ActiveDoc;
        SelectMeshBody();
    }

    public void SelectGraphicsBody()
    {
        Debugger.Break();
        Debug.Print("Getting the graphics body, Graphic (Closed) -1...");
        bool boolstatus;
        boolstatus = swModelDoc.Extension.SelectByID2("Graphic (Closed) -1", "MESH BODY
FEATURE", -0.0932538825377272d, 0.091d, 0.033742378078639d, false, 0, null, 0);
        SelectionMgr swSelMgr;
        swSelMgr = (SelectionMgr)swModelDoc.SelectionManager;
        int lSelType;
        lSelType = swSelMgr.GetSelectedObjectType3(1, -1);
        if (lSelType == (int)swSelectType_e.swSelGRAPHICSBODY)
        {
            Debug.Print("lSelType = swSelGRAPHICSBODY");
        }
    }

```

```

Feature swFeature;
swFeature = (Feature)swSelMgr.GetSelectedObject6(1, -1);
if (swFeature == null)
{
    Debug.Print("swFeature is Nothing");
}
else if (lSelType == (int)swSelectType_e.swSelGRAPHICSBODY)
{
    Debug.Print("Select GraphicsBody success");
    Body2 swBody;
    swBody = (Body2)swFeature.GetSpecificFeature2();
    if (swBody == null)
    {
        Debug.Print("swBody Is Nothing");
    }
    else
    {
        Debug.Print("Getting Body success");
        int lBodyType;
        lBodyType = swBody.GetType();
        if (lBodyType == (int)swBodyType_e.swGraphicsBody)
        {
            GraphicsBody swGraphicsBody;
            swGraphicsBody = (GraphicsBody)swBody.GetGraphicsBody();
            if (swGraphicsBody == null)
            {
                Debug.Print("Graphics body object is Nothing");
            }
            else
            {
                Debug.Print("GetGraphicsBody success");
                Body2 swBodyFromGraphics;
                swBodyFromGraphics = (Body2)swGraphicsBody.GetBody();
                if (swBodyFromGraphics == null)
                {
                    Debug.Print("swGraphicsBody.GetBody failed");
                }
                else
                {
                    Debug.Print("swGraphicsBody.GetBody success");
                }
            }
        }
    }
}
}

```

body...");

```

Debug.Print("Selecting facet, facet edge, and facet vertex objects in graphics

```

```

        swModelDoc.ClearSelection2(true);
        boolstatus = swModelDoc.Extension.SelectByID2("Unknown", "MESHFACETREF",
-0.130140285581291d, 0.0909999999999798d, 0.0526687725669035d, false, 0, null, 0);
        lSelType = swSelMgr.GetSelectedObjectType3(1, -1);
        if (lSelType == (int)swSelectType_e.swSelFACETS)
        {
            Debug.Print("Facet selected");
        }
        else
        {
            Debug.Print("Object selected not of type facet");
        }

        swModelDoc.ClearSelection2(true);
        boolstatus = swModelDoc.Extension.SelectByID2("Unknown", "MESHFINREF",
-0.0701858537194084d, 0.0909999999999798d, 0.0513894806662165d, false, 0, null, 0);
        lSelType = swSelMgr.GetSelectedObjectType3(1, -1);
        if (lSelType == (int)swSelectType_e.swSelMESHFACETEDGES)
        {
            Debug.Print("Facet edge selected");
        }
        else
        {
            Debug.Print("Object selected not of type facet edge");
        }

        swModelDoc.ClearSelection2(true);
        boolstatus = swModelDoc.Extension.SelectByID2("Unknown", "MESHVERTEXREF",
-0.117127148857435d, 0.0909999999999513d, 0.099920771345694d, false, 0, null, 0);
        lSelType = swSelMgr.GetSelectedObjectType3(1, -1);
        if (lSelType == (int)swSelectType_e.swSelMESHFACETVERTICES)
        {
            Debug.Print("Facet vertex selected");
        }
        else
        {
            Debug.Print("Object selected not of type facet vertex");
        }

        Debug.Print("");
    }

    public void SelectMeshBody()
    {
        Debugger.Break();
        Debug.Print("Getting mesh body, Surface-Imported5...");
        bool boolstatus;
        boolstatus = swModelDoc.Extension.SelectByID2("Surface-Imported5", "SURFACEBODY",
-0.0955822368965755d, 0.09100000000000082d, 0.0336061875443647d, false, 0, null, 0);

```

```

SelectionMgr swSelMgr;
swSelMgr = (SelectionMgr)swModelDoc.SelectionManager;
int lSelType;
lSelType = swSelMgr.GetSelectedObjectType3(1, -1);
if (lSelType == (int)swSelectType_e.swSelSURFACEBODIES)
{
    Body2 swBody;
    swBody = (Body2)swSelMgr.GetSelectedObject6(1, -1);
    int lBodyType;
    lBodyType = swBody.GetType();
    if (lBodyType == (int)swBodyType_e.swMeshBody)
    {
        Debug.Print("Select MeshBody success");
        MeshBody swMeshBody;
        swMeshBody = (MeshBody)swBody.GetMeshBody();
        if (swMeshBody == null)
        {
            Debug.Print("Mesh body object is Nothing");
        }
        else
        {
            Debug.Print("Mesh body retrieved");
            Body2 swMBody;
            swMBody = (Body2)swMeshBody.GetBody();
            if (swMBody == null)
            {
                Debug.Print("swMeshBody.GetBody failed");
            }
            else
            {
                Debug.Print("swMeshBody.GetBody success");
            }
        }
    }
}

Debug.Print("");
}

public void TestBodies()
{
    Debugger.Break();
    Debug.Print("Testing facets, facet edges, and facet vertexes...");
    bool bFilter;
    bFilter = swApp.GetSelectionFilter((int)swSelectType_e.swSelMESHFACETEDGES);
    swApp.SetSelectionFilter((int)swSelectType_e.swSelMESHFACETEDGES, bFilter == false);
    bFilter = swApp.GetSelectionFilter((int)swSelectType_e.swSelMESHFACETEDGES);
    int[] vFilters;

```

```

vFilters = (int[])swApp.GetSelectionFilters();
int j;
if (vFilters.Length > 0)
{
    var loopTo = vFilters.GetUpperBound(0);
    for (j = 0; j <= loopTo; j++)
    {
        int lCurFilter;
        lCurFilter = vFilters[j];
        if (lCurFilter == (int)swSelectType_e.swSelGRAPHICSBODY)
        {
            Debug.Print("Selection filter is swSelGRAPHICSBODY");
        }
        else if (lCurFilter == (int)swSelectType_e.swSelFACETS)
        {
            Debug.Print("Selection filter is swSelFACETS");
        }
        else if (lCurFilter == (int)swSelectType_e.swSelMESHFACETEDGES)
        {
            Debug.Print("Selection filter is swSelMESHFACETEDGES");
        }
        else if (lCurFilter == (int)swSelectType_e.swSelMESHFACETVERTICES)
        {
            Debug.Print("Selection filter is swSelMESHFACETVERTICES");
        }
        else
        {
            Debug.Print("Selection filter is other than facet/facet edge/facet
vertex");
        }
    }
}

var lSelFilters = new int[1];
object vSelFilters;
lSelFilters[0] = (int)swSelectType_e.swSelMESHFACETVERTICES;
// lSelFilters(1) = swSelectType_e.swSelFACETS
vSelFilters = lSelFilters;
swApp.SetSelectionFilters(vSelFilters, false);
bool bApplySelFilters;
bApplySelFilters = swApp.GetApplySelectionFilter();
bool bOppSelFilter;
bOppSelFilter = bApplySelFilters == false;
swApp.SetApplySelectionFilter(bOppSelFilter);
swApp.SetApplySelectionFilter(false);
swModelDoc = (ModelDoc2)swApp.ActiveDoc;
PartDoc swPartDoc;
swPartDoc = (PartDoc)swModelDoc;

```

```

object[] vBodies;
vBodies = (object[])swPartDoc.GetBodies2(-1, false);
SelectionMgr swSelMgr;
swSelMgr = (SelectionMgr)swModelDoc.SelectionManager;
int lSelType;
Body2 swCurBod;
if (vBodies.Length > 0)
{
    int i;
    var loopTo1 = vBodies.GetUpperBound(0);
    for (i = 0; i <= loopTo1; i++)
    {
        Debug.Print("");
        swCurBod = (Body2)vBodies[i];
        int lCurBodyType;
        lCurBodyType = swCurBod.GetType();
        if (lCurBodyType == (int)swBodyType_e.swMeshBody)
        {
            Debug.Print("Testing mesh body");
        }
        else if (lCurBodyType == (int)swBodyType_e.swGraphicsBody)
        {
            Debug.Print("Testing graphics body");
        }
        else
        {
            Debug.Print("Some other body type");
        }

        swCurBod.Select2(false, null);
        swCurBod.DeSelect();
        bool bIsVisible;
        bIsVisible = swCurBod.Visible;
        Debug.Print("Visible is " + bIsVisible);
        string sBodyName;
        sBodyName = swCurBod.Name;
        Debug.Print("Name is \"" + sBodyName + "\"");
        bool bIsMesh;
        bIsMesh = swCurBod.IsMeshBody();
        Debug.Print("IsMeshBody returned " + bIsMesh);
        bool bIsGraphics;
        bIsGraphics = swCurBod.IsGraphicsBody();
        Debug.Print("IsGraphicsBody method returned " + bIsGraphics);
        MeshBody swThisMeshBody;
        swThisMeshBody = (MeshBody)swCurBod.GetMeshBody(); //
        if (swThisMeshBody == null)
        {
            Debug.Print("");
        }
    }
}

```

```

    }
    else
    {
        Body2 swThisBodyFromMesh;
        swThisBodyFromMesh = (Body2)swThisMeshBody.GetBody();
        if (swThisBodyFromMesh is object)
        {
            Debug.Print("swThisMeshBody.GetBody returned an IBody2.");
        }
        else
        {
            Debug.Print("swThisMeshBody.GetBody returned Nothing.");
        }

        Debug.Print("Getting facet, facet edge, and facet vertex objects in this
mesh body...");

        int lFacetCount;
        lFacetCount = swThisMeshBody.GetFacetCount();
        Debug.Print("Facet count is " + lFacetCount);
        object[] vMFacets;
        vMFacets = (object[])swThisMeshBody.GetFacets();
        int lMFacetIndex;
        var loopTo2 = vMFacets.GetUpperBound(0);
        for (lMFacetIndex = 0; lMFacetIndex <= loopTo2; lMFacetIndex++)
        {
            Debug.Print("");
            Debug.Print("Facet " + (lMFacetIndex + 1));
            Facet swCurMFacet;
            swCurMFacet = (Facet)vMFacets[lMFacetIndex];
            Body2 swBodyFromCurMFacet;
            swBodyFromCurMFacet = (Body2)swCurMFacet.GetBody();
            object[] vEdges;
            object[] vVerts;
            vEdges = (object[])swCurMFacet.GetFacetEdges();
            int lEdgesIndex;
            var loopTo3 = vEdges.GetUpperBound(0);
            for (lEdgesIndex = 0; lEdgesIndex <= loopTo3; lEdgesIndex++)
            {
                Edge swCurEdge;
                swCurEdge = (Edge)vEdges[lEdgesIndex];
                if (swCurEdge == null)
                {
                    Debug.Print("Edge object is Nothing");
                }
                else
                {
                    Debug.Print("Edge retrieved");
                }
            }
        }
    }
}

```

```

    }

    vVerts = (object[])swCurMFacet.GetFacetVertices();
    int lVertsIndex;
    var loopTo4 = vVerts.GetUpperBound(0);
    for (lVertsIndex = 0; lVertsIndex <= loopTo4; lVertsIndex++)
    {
        Vertex swCurVert;
        swCurVert = (Vertex)vVerts[lVertsIndex];
        if (swCurVert == null)
        {
            Debug.Print("Vertex object is Nothing");
        }
        else
        {
            Debug.Print("Vertex retrieved");
        }
    }

    swCurMFacet.Select(false, null);
    lSelType = swSelMgr.GetSelectedObjectType3(1, -1);
    if (lSelType == (int)swSelectType_e.swSelFACETS)
    {
        Debug.Print("Facet selected");
    }
    else
    {
        Debug.Print("Object not of type Facet");
    }
}

}

GraphicsBody swThisGraphicsBody;
swThisGraphicsBody = (GraphicsBody)swCurBod.GetGraphicsBody();
if (swThisGraphicsBody == null)
{
    Debug.Print("");
}
else
{
    Body2 swThisBodyFromGraphics;
    swThisBodyFromGraphics = (Body2)swThisGraphicsBody.GetBody();
    if (swThisBodyFromGraphics is object)
    {
        Debug.Print("swThisGraphicsBody.GetBody returned an IBody2.");
    }
    else
    {

```

```

        Debug.Print("swThisGraphicsBody.GetBody returned Nothing.");
    }

    Debug.Print("Getting facet, facet edge, and facet vertex objects in this
graphics body...");

    int lGFacetCount;
    lGFacetCount = swThisGraphicsBody.GetFacetCount();
    Debug.Print("Facet count is " + lGFacetCount);
    object[] vFacets;
    vFacets = (object[])swThisGraphicsBody.GetFacets();
    int lFacetIndex;
    var loopTo5 = vFacets.GetUpperBound(0);
    for (lFacetIndex = 0; lFacetIndex <= loopTo5; lFacetIndex++)
    {
        Debug.Print("");
        Debug.Print("Facet " + (lFacetIndex + 1));
        Facet swFacet;
        swFacet = (Facet)vFacets[lFacetIndex];
        Body2 swFacetBody;
        swFacetBody = (Body2)swFacet.GetBody();
        if (swFacetBody == null)
        {
            Debug.Print("swFacet.GetBody failed");
        }
        else
        {
            Debug.Print("swFacet.GetBody returned an IBody2");
        }

        object[] vGEdges;
        vGEdges =(object[]) swFacet.GetFacetEdges();
        int lFacetEdgesCount;
        lFacetEdgesCount = vGEdges.GetUpperBound(0) + 1;
        int lFacetEdgesIndex;
        var loopTo6 = vGEdges.GetUpperBound(0);
        for (lFacetEdgesIndex = 0; lFacetEdgesIndex <= loopTo6;
lFacetEdgesIndex++)
        {
            Edge swFacetEdge;
            swFacetEdge = (Edge)vGEdges[lFacetEdgesIndex];
            if (swFacetEdge == null)
            {
                Debug.Print("Edge object is Nothing");
            }
            else
            {
                Debug.Print("Edge retrieved");
            }
        }
    }

```

```

        object[] vGVerts;
        vGVerts = (object[])swFacet.GetFacetVertices();
        int lFacetVertsCount;
        lFacetVertsCount = vGVerts.GetUpperBound(0) + 1;
        int lFacetVertsIndex;
        var loopTo7 = vGVerts.GetUpperBound(0);
        for (lFacetVertsIndex = 0; lFacetVertsIndex <= loopTo7;
lFacetVertsIndex++)
        {
            Vertex swFacetVert;
            swFacetVert = (Vertex)vGVerts[lFacetVertsIndex];
            if (swFacetVert == null)
            {
                Debug.Print("Vertex object is Nothing");
            }
            else
            {
                Debug.Print("Vertex retrieved");
            }
        }

        swFacet.Select(false, null);
        lSelType = swSelMgr.GetSelectedObjectType3(1, -1);
        if (lSelType == (int)swSelectType_e.swSelFACETS)
        {
            Debug.Print("Facet selected");
        }
        else
        {
            Debug.Print("Object not of type Facet");
        }
    }
}

    }

    Debug.Print("");
}

// The SldWorks swApp variable is pre-assigned for you.
public SldWorks swApp;
}
}

```

Inspect Facets of Graphics and Mesh Bodies Example (VB.NET)

This example shows how to inspect facets, facet edges, and facet vertexes after importing graphics and mesh bodies.

```
'=====
```

```
'Preconditions:
```

```
'1. Copy Public_Documents\SOLIDWORKS\SOLIDWORKS 2022\samples\tutorial\api\Part1.PLY to c:\temp.
```

```
'2. Open the Immediate window.
```

```
'Postconditions:
```

```
'1. Press F5 five times, inspecting the Immediate window after each press.
```

```
'2. The part is imported first as a graphics body and then as a mesh body.
```

```
'3. The macro tests facets, facet edges, and facet vertexes after each import.
```

```
'=====
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Dim swModelDoc As ModelDoc2
```

```
    Sub main()
```

```
        swModelDoc = Nothing
```

```
        TestGraphicsBody()
```

```
        TestBodies()
```

```
        swModelDoc = Nothing
```

```
        swApp.CloseDoc("Part1")
```

```
        TestMeshBody()
```

```
        TestBodies()
```

```
        swModelDoc = Nothing
```

```
        swApp.CloseDoc("Part1")
```

End Sub

Sub TestGraphicsBody()

Debug.Print("Importing Part1.PLY as a graphics body...")

Debug.Print("")

Dim boolstatus As Boolean

boolstatus =

swApp.**SetUserPreferenceIntegerValue**(swUserPreferenceIntegerValue_e.swImportStlVrmlModelType,
swImportStlVrmlModelType_e.swImportStlVrmlModelType_Graphics)

boolstatus =

swApp.**SetUserPreferenceIntegerValue**(swUserPreferenceIntegerValue_e.swImportStlVrmlModelType, 0)

boolstatus = swApp.**LoadFile2**("c:\temp\Part1.PLY", "r")

swModelDoc = swApp.ActiveDoc

SelectGraphicsBody()

End Sub

Sub TestMeshBody()

Debug.Print("Importing Part1.PLY as a mesh body...")

Debug.Print("")

Dim boolstatus As Boolean

boolstatus =

swApp.**SetUserPreferenceIntegerValue**(swUserPreferenceIntegerValue_e.swImportStlVrmlModelType,
swImportStlVrmlModelType_e.swImportStlVrmlModelType_Surface)

boolstatus =

swApp.**SetUserPreferenceIntegerValue**(swUserPreferenceIntegerValue_e.swImportStlVrmlModelType, 1)

swApp.**SetUserPreferenceToggle**(swUserPreferenceToggle_e.swVrmlStlImportAsPSMesh, True)

boolstatus = swApp.**LoadFile2**("c:\temp\Part1.PLY", "r")

swModelDoc = swApp.ActiveDoc

SelectMeshBody()

End Sub

Sub SelectGraphicsBody()

Stop

Debug.Print("Getting the graphics body, Graphic (Closed) -1...")

Dim boolstatus As Boolean

boolstatus = swModelDoc.Extension.SelectByID2("Graphic (Closed) -1", "MESH BODY FEATURE",
-0.0932538825377272, 0.091, 0.033742378078639, False, 0, Nothing, 0)

Dim swSelMgr As SelectionMgr

```

swSelMgr = swModelDoc.SelectionManager

Dim lSelType As Integer
lSelType = swSelMgr.GetSelectedObjectType3(1, -1)
If lSelType = swSelectType_e.swSelGRAPHICSBODY Then
    Debug.Print("lSelType = swSelGRAPHICSBODY")
End If

Dim swFeature As Feature
swFeature = swSelMgr.GetSelectedObject6(1, -1)

If swFeature Is Nothing Then
    Debug.Print("swFeature is Nothing")
ElseIf lSelType = swSelectType_e.swSelGRAPHICSBODY Then
    Debug.Print("Select GraphicsBody success")
    Dim swBody As Body2
    swBody = swFeature.GetSpecificFeature2
    If swBody Is Nothing Then
        Debug.Print("swBody Is Nothing")
    Else
        Debug.Print("Getting Body success")
        Dim lBodyType As Integer
        lBodyType = swBody.GetType

        If lBodyType = swBodyType_e.swGraphicsBody Then
            Dim swGraphicsBody As GraphicsBody
            swGraphicsBody = swBody.GetGraphicsBody
            If swGraphicsBody Is Nothing Then
                Debug.Print("Graphics body object is Nothing")
            Else
                Debug.Print("GetGraphicsBody success")
                Dim swBodyFromGraphics As Body2
                swBodyFromGraphics = swGraphicsBody.GetBody
                If swBodyFromGraphics Is Nothing Then
                    Debug.Print("swGraphicsBody.GetBody failed")
                Else
                    Debug.Print("swGraphicsBody.GetBody success")
                End If
            End If
        End If
    End If
End If

Debug.Print("Selecting facet, facet edge, and facet vertex objects in graphics body...")

swModelDoc.ClearSelection2(True)
boolstatus = swModelDoc.Extension.SelectByID2("Unknown", "MESHFACETREF",
-0.130140285581291, 0.0909999999999798, 0.0526687725669035, False, 0, Nothing, 0)

```

```

lSelType = swSelMgr.GetSelectedObjectType3(1, -1)
If lSelType = swSelectType_e.swSelFACETS Then
    Debug.Print("Facet selected")
Else
    Debug.Print("Object selected not of type facet")
End If

swModelDoc.ClearSelection2(True)
boolstatus = swModelDoc.Extension.SelectByID2("Unknown", "MESHFINREF",
-0.0701858537194084, 0.0909999999999798, 0.0513894806662165, False, 0, Nothing, 0)
lSelType = swSelMgr.GetSelectedObjectType3(1, -1)
If lSelType = swSelectType_e.swSelMESHFACETEDGES Then
    Debug.Print("Facet edge selected")
Else
    Debug.Print("Object selected not of type facet edge")
End If

swModelDoc.ClearSelection2(True)
boolstatus = swModelDoc.Extension.SelectByID2("Unknown", "MESHVERTEXREF",
-0.117127148857435, 0.0909999999999513, 0.099920771345694, False, 0, Nothing, 0)
lSelType = swSelMgr.GetSelectedObjectType3(1, -1)
If lSelType = swSelectType_e.swSelMESHFACETVERTICES Then
    Debug.Print("Facet vertex selected")
Else
    Debug.Print("Object selected not of type facet vertex")
End If

Debug.Print("")
End Sub

Sub SelectMeshBody()
    Stop
    Debug.Print("Getting mesh body, Surface-Imported5...")
    Dim boolstatus As Boolean

    boolstatus = swModelDoc.Extension.SelectByID2("Surface-Imported5", "SURFACEBODY",
-0.0955822368965755, 0.0910000000000082, 0.0336061875443647, False, 0, Nothing, 0)

    Dim swSelMgr As SelectionMgr
    swSelMgr = swModelDoc.SelectionManager

    Dim lSelType As Integer
    lSelType = swSelMgr.GetSelectedObjectType3(1, -1)
    If lSelType = swSelectType_e.swSelSURFACEBODIES Then
        Dim swBody As Body2
        swBody = swSelMgr.GetSelectedObject6(1, -1)
        Dim lBodyType As Integer
        lBodyType = swBody.GetType
        If lBodyType = swBodyType_e.swMeshBody Then

```

```

        Debug.Print("Select MeshBody success")
        Dim swMeshBody As MeshBody
        swMeshBody = swBody.GetMeshBody
        If swMeshBody Is Nothing Then
            Debug.Print("Mesh body object is Nothing")
        Else
            Debug.Print("Mesh body retrieved")
            Dim swMBody As Body2
            swMBody = swMeshBody.GetBody
            If swMBody Is Nothing Then
                Debug.Print("swMeshBody.GetBody failed")
            Else
                Debug.Print("swMeshBody.GetBody success")
            End If
        End If
    End If
End If

Debug.Print("")
End Sub

Sub TestBodies()
    Stop
    Debug.Print("Testing facets, facet edges, and facet vertexes...")

    Dim bFilter As Boolean
    bFilter = swApp.GetSelectionFilter(swSelectType_e.swSelMESHFACETEDGES)
    swApp.SetSelectionFilter(swSelectType_e.swSelMESHFACETEDGES, (bFilter = False))
    bFilter = swApp.GetSelectionFilter(swSelectType_e.swSelMESHFACETEDGES)

    Dim vFilters() As Integer
    vFilters = swApp.GetSelectionFilters

    Dim j As Integer

    If vFilters.Length > 0 Then
        For j = 0 To UBound(vFilters)
            Dim lCurFilter As Integer
            lCurFilter = vFilters(j)

            If lCurFilter = swSelectType_e.swSelGRAPHICSBODY Then
                Debug.Print("Selection filter is swSelGRAPHICSBODY")
            ElseIf lCurFilter = swSelectType_e.swSelFACETS Then
                Debug.Print("Selection filter is swSelFACETS")
            ElseIf lCurFilter = swSelectType_e.swSelMESHFACETEDGES Then
                Debug.Print("Selection filter is swSelMESHFACETEDGES")
            ElseIf lCurFilter = swSelectType_e.swSelMESHFACETVERTICES Then

```

```

        Debug.Print("Selection filter is swSelMESHFACETVERTICES")
    Else
        Debug.Print("Selection filter is other than facet/facet edge/facet vertex")
    End If
Next j
End If

Dim lSelFilters(0) As Integer
Dim vSelFilters As Object
lSelFilters(0) = swSelectType_e.swSelMESHFACETVERTICES
'lSelFilters(1) = swSelectType_e.swSelFACETS
vSelFilters = lSelFilters
swApp.SetSelectionFilters(vSelFilters, False)

Dim bApplySelFilters As Boolean
bApplySelFilters = swApp.GetApplySelectionFilter
Dim bOppSelFilter As Boolean
bOppSelFilter = (bApplySelFilters = False)
swApp.SetApplySelectionFilter(bOppSelFilter)

swApp.SetApplySelectionFilter(False)

swModelDoc = swApp.ActiveDoc
Dim swPartDoc As PartDoc
swPartDoc = swModelDoc

Dim vBodies() As Object
vBodies = swPartDoc.GetBodies2(-1, False)

Dim swSelMgr As SelectionMgr
swSelMgr = swModelDoc.SelectionManager
Dim lSelType As Integer

Dim swCurBod As Body2
If vBodies.Length > 0 Then
    Dim i As Integer
    For i = 0 To UBound(vBodies)
        Debug.Print("")

        swCurBod = vBodies(i)

        Dim lCurBodyType As Integer
        lCurBodyType = swCurBod.GetType
        If lCurBodyType = swBodyType_e.swMeshBody Then
            Debug.Print("Testing mesh body")
        ElseIf lCurBodyType = swBodyType_e.swGraphicsBody Then
            Debug.Print("Testing graphics body")
        End If
    Next i
End If

```

```

Else
    Debug.Print("Some other body type")
End If

swCurBod.Select2(False, Nothing)

swCurBod.DeSelect()

Dim bIsVisible As Boolean
bIsVisible = swCurBod.Visible
Debug.Print("Visible is " & bIsVisible)

Dim sBodyName As String
sBodyName = swCurBod.Name
Debug.Print("Name is "" & sBodyName & """)

Dim bIsMesh As Boolean
bIsMesh = swCurBod.IsMeshBody
Debug.Print("IsMeshBody returned " & bIsMesh)

Dim bIsGraphics As Boolean
bIsGraphics = swCurBod.IsGraphicsBody
Debug.Print("IsGraphicsBody method returned " & bIsGraphics)

Dim swThisMeshBody As MeshBody
swThisMeshBody = swCurBod.GetMeshBody
If swThisMeshBody Is Nothing Then
    Debug.Print("")
Else
    Dim swThisBodyFromMesh As Body2
    swThisBodyFromMesh = swThisMeshBody.GetBody
    If Not swThisBodyFromMesh Is Nothing Then
        Debug.Print("swThisMeshBody.GetBody returned an IBody2.")
    Else
        Debug.Print("swThisMeshBody.GetBody returned Nothing.")
    End If

    Debug.Print("Getting facet, facet edge, and facet vertex objects in this mesh
body...")

    Dim lFacetCount As Integer
    lFacetCount = swThisMeshBody.GetFacetCount
    Debug.Print("Facet count is " & lFacetCount)

    Dim vMFacets() As Object
    vMFacets = swThisMeshBody.GetFacets
    Dim lMFacetIndex As Integer
    For lMFacetIndex = 0 To UBound(vMFacets)
        Debug.Print("")
    
```

```

        Debug.Print("Facet " & lMFacetIndex + 1)
        Dim swCurMFacet As Facet
        swCurMFacet = vMFacets(lMFacetIndex)
        Dim swBodyFromCurMFacet As Body2
        swBodyFromCurMFacet = swCurMFacet.GetBody
        Dim vEdges() As Object
        Dim vVerts() As Object
        vEdges = swCurMFacet.GetFacetEdges
        Dim lEdgesIndex As Integer
        For lEdgesIndex = 0 To UBound(vEdges)
            Dim swCurEdge As Edge
            swCurEdge = vEdges(lEdgesIndex)
            If swCurEdge Is Nothing Then
                Debug.Print("Edge object is Nothing")
            Else
                Debug.Print("Edge retrieved")
            End If

        Next lEdgesIndex
        vVerts = swCurMFacet.GetFacetVertices
        Dim lVertsIndex As Integer
        For lVertsIndex = 0 To UBound(vVerts)
            Dim swCurVert As Vertex
            swCurVert = vVerts(lVertsIndex)
            If swCurVert Is Nothing Then
                Debug.Print("Vertex object is Nothing")
            Else
                Debug.Print("Vertex retrieved")
            End If

        Next lVertsIndex
        swCurMFacet.Select(False, Nothing)
        lSelType = swSelMgr.GetSelectedObjectType3(1, -1)
        If lSelType = swSelectType_e.swSelFACETS Then
            Debug.Print("Facet selected")
        Else
            Debug.Print("Object not of type Facet")
        End If
    Next lMFacetIndex

End If

Dim swThisGraphicsBody As GraphicsBody
swThisGraphicsBody = swCurBod.GetGraphicsBody
If swThisGraphicsBody Is Nothing Then
    Debug.Print("")
Else
    Dim swThisBodyFromGraphics As Body2
    swThisBodyFromGraphics = swThisGraphicsBody.GetBody

```

```

If Not swThisBodyFromGraphics Is Nothing Then
    Debug.Print("swThisGraphicsBody.GetBody returned an IBody2.")
Else
    Debug.Print("swThisGraphicsBody.GetBody returned Nothing.")
End If
Debug.Print("Getting facet, facet edge, and facet vertex objects in this
graphics body...")

```

```

Dim lGFacetCount As Integer
lGFacetCount = swThisGraphicsBody.GetFacetCount
Debug.Print("Facet count is " & lGFacetCount)

```

```

Dim vFacets() As Object
vFacets = swThisGraphicsBody.GetFacets
Dim lFacetIndex As Integer
For lFacetIndex = 0 To UBound(vFacets)
    Debug.Print("")
    Debug.Print("Facet " & lFacetIndex + 1)
    Dim swFacet As Facet
    swFacet = vFacets(lFacetIndex)
    Dim swFacetBody As Body2
    swFacetBody = swFacet.GetBody
    If swFacetBody Is Nothing Then
        Debug.Print("swFacet.GetBody failed")
    Else
        Debug.Print("swFacet.GetBody returned an IBody2")
    End If

```

```

Dim vGEdges() As Object
vGEdges = swFacet.GetFacetEdges
Dim lFacetEdgesCount As Integer
lFacetEdgesCount = UBound(vGEdges) + 1
Dim lFacetEdgesIndex As Integer
For lFacetEdgesIndex = 0 To UBound(vGEdges)
    Dim swFacetEdge As Edge
    swFacetEdge = vGEdges(lFacetEdgesIndex)
    If swFacetEdge Is Nothing Then
        Debug.Print("Edge object is Nothing")
    Else
        Debug.Print("Edge retrieved")
    End If
Next lFacetEdgesIndex

```

```

Dim vGVerts() As Object
vGVerts = swFacet.GetFacetVertices
Dim lFacetVertsCount As Integer
lFacetVertsCount = UBound(vGVerts) + 1
Dim lFacetVertsIndex As Integer

```

```

        For lFacetVertsIndex = 0 To UBound(vGVerts)
            Dim swFacetVert As Vertex
            swFacetVert = vGVerts(lFacetVertsIndex)
            If swFacetVert Is Nothing Then
                Debug.Print("Vertex object is Nothing")
            Else
                Debug.Print("Vertex retrieved")
            End If

        Next lFacetVertsIndex

        swFacet.Select(False, Nothing)
        lSelType = swSelMgr.GetSelectedObjectType3(1, -1)
        If lSelType = swSelectType_e.swSelFACETS Then
            Debug.Print("Facet selected")
        Else
            Debug.Print("Object not of type Facet")
        End If

    Next lFacetIndex

End If
Next i
End If
Debug.Print("")

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
End Class

```

Is_Feature_Suppressed_in_Specified_Configurations_Example_VB (Topic)

SOLIDWORKS API Help

Is Feature Suppressed in Configuration Example (VBA)

This example shows how to find out if a feature is suppressed in the specified configurations.

```
' -----  
' Preconditions:  
' 1. Open a part or assembly with one configuration.  
' 2. Select a feature.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Finds out if the selected feature is  
'    suppressed in the configuration.  
' 2. Examine the Immediate window.  
' -----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swFeat As SldWorks.Feature  
    Dim vConfNameArr As Variant  
    Dim vSuppStateArr As Variant  
    Dim i As Long  
    Dim bRet As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)  
  
    vConfNameArr = swModel.GetConfigurationNames  
    vSuppStateArr = swFeat.IsSuppressed2(swThisConfiguration, vConfNameArr)  
  
    Debug.Print "File = " & swModel.GetPathName  
    Debug.Print " " & swFeat.Name  
    For i = 0 To UBound(vConfNameArr)  
        Debug.Print " " & vConfNameArr(i) & " --> " & vSuppStateArr(i)  
    Next i  
  
End Sub
```

Is Selected Feature a Boundary Box Sketch (C#)

This example shows how to determine if a sketch is a boundary box sketch.

```
//-----  
// Preconditions:  
// 1. Open public_documents\samples\tutorial\api\2012-sm.sldprt.  
// 2. Open the Immediate window.  
//  
// Postconditions:  
// 1. Unsuppresses the Flat-Pattern1 feature.  
// 2. Selects the Flat-Pattern1's boundary box sketch feature.  
// 3. Gets whether the selected sketch is a boundary box sketch.  
// 4. Examine the Immediate window.  
//  
// NOTE: Because the part is used elsewhere, do not save changes.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System;  
using System.Diagnostics;  
  
namespace IsBoundaryBoxSketchSketchCSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        public void Main()  
        {  
            ModelDoc2 swModel;  
            ModelDocExtension swModelDocExt;  
            SelectionMgr swSelMgr;  
            Feature swFeature;  
            Sketch swSketch;  
            bool status = false;  
  
            // Open a sheet metal part  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
  
            // Select the flat-pattern feature  
            swModelDocExt = (ModelDocExtension)swModel.Extension;  
            status = swModelDocExt.SelectByID2("Flat-Pattern1", "BODYFEATURE", 0, 0, 0, false, 0,  
null, 0);  
  
            // Unsuppress the flat-pattern feature  
            status = swModel.EditUnsuppress2();  
            swModel.ClearSelection2(true);  
  
            // Select the flat-pattern feature's boundary box feature  
            status = swModelDocExt.SelectByID2("Bounding-Box1", "SKETCH", 0, 0, 0, false, 0, null,  
0);  
  
            swSelMgr = (SelectionMgr)swModel.SelectionManager;  
            swFeature = (Feature)swSelMgr.GetSelectedObject6(1, -1);  
            swSketch = (Sketch)swFeature.GetSpecificFeature2();  
  
            // Print to the Immediate window if the just-selected feature is a boundary box  
            Debug.Print("Selected sketch a boundary box sketch? " +  
swSketch.IsBoundaryBoxSketch());  
        }  
  
        /// <summary>  
        /// The SldWorks swApp variable is pre-assigned for you.  
        /// </summary>  
        public SldWorks swApp;
```

} }

Is_Selected_Feature_a_Boundary_Box_Sketch_Example_VB (Topic)

SOLIDWORKS API Help

Is Selected Feature a Boundary Box Sketch (VBA)

This example shows how to determine if a sketch is a boundary box sketch.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\2012-sm.sldprt.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Unsuppresses the Flat-Pattern1 feature.  
' 2. Selects the Flat-Pattern1's boundary box sketch feature.  
' 3. Gets whether the selected sketch is a boundary box sketch.  
' 4. Examine the Immediate window.  
'  
' NOTE: Because the part is used elsewhere, do not save changes.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swFeature As SldWorks.Feature  
Dim swSketch As SldWorks.Sketch  
Dim status As Boolean  
  
Sub main()  
    Set swApp = Application.SldWorks  
  
    ' Open a sheet metal part  
    Set swModel = swApp.ActiveDoc  
  
    ' Select the flat-pattern feature  
    Set swModelDocExt = swModel.Extension  
    status = swModelDocExt.SelectByID("Flat-Pattern1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)  
  
    ' Unsuppress the flat-pattern feature  
    status = swModel.EditUnsuppress2  
    swModel.ClearSelection2 True  
  
    ' Select the flat-pattern feature's boundary box feature  
    status = swModelDocExt.SelectByID2("Bounding-Box1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)  
    Set swSelMgr = swModel.SelectionManager  
    Set swFeature = swSelMgr.GetSelectedObject6(1, -1)  
    Set swSketch = swFeature.GetSpecificFeature2  
  
    ' Print to the Immediate window if the just-selected feature is a boundary box  
    Debug.Print "Selected feature a boundary box sketch? " & swSketch.IsBoundaryBoxSketch  
  
End Sub
```

Is Selected Feature a Boundary Box Sketch (VB.NET)

This example shows how to determine if a sketch is a boundary box sketch.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\2012-sm.sldprt.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Unsuppresses the Flat-Pattern1 feature.
' 2. Selects the Flat-Pattern1's boundary box sketch feature.
' 3. Gets whether the selected sketch is a boundary box sketch.
' 4. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System

Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelMgr As SelectionMgr
        Dim swFeature As Feature
        Dim swSketch As Sketch
        Dim status As Boolean

        ' Open a sheet metal part
        swModel = swApp.ActiveDoc

        ' Select the flat-pattern feature
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID2("Flat-Pattern1", "BODYFEATURE", 0, 0, 0, False, 0,
Nothing, 0)

        ' Unsuppress the flat-pattern feature
        status = swModel.EditUnsuppress2
        swModel.ClearSelection2(True)

        ' Select the flat-pattern feature's boundary box feature
        status = swModelDocExt.SelectByID2("Bounding-Box1", "SKETCH", 0, 0, 0, False, 0, Nothing,
0)

        swSelMgr = swModel.SelectionManager
        swFeature = swSelMgr.GetSelectedObject6(1, -1)
        swSketch = swFeature.GetSpecificFeature2

        ' Print to the Immediate window if the just-selected feature is a boundary box
        Debug.Print("Selected sketch a boundary box sketch? " & swSketch.IsBoundaryBoxSketch)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Is There a Projection Arrow and Is It Visible Example (C#)

This example shows how to find out if:

- the selected drawing view is a projected view
- a projection arrow exists in the projected view
- the projected arrow is visible

```
//-----  
// Preconditions:  
// 1. Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.  
// 2. Double-click the projected view (i.e., the upper-right drawing view).  
// 3. In the PropertyManager page, click the Arrow check box and  
//    type a label for the projection arrow.  
// 4. Click OK to close the PropertyManager page.  
//  
// Postconditions:  
// 1. Creates a visible projection arrow for Drawing View2, which is  
//    a projected view.  
// 2. Examine the graphics area and Immediate window.  
//  
// NOTE: Because this drawing document is used elsewhere, do not save  
// changes.  
//-----  
  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System;  
using System.Diagnostics;  
  
namespace Visibleprojectionarrowcsharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        public void Main()  
        {  
            ModelDoc2 swModel;  
            ModelDocExtension swModelDocExt;  
            DrawingDoc swDrawing;  
            View swView;  
            ProjectionArrow swProjectionArrow;  
            int typeDrawingView;  
            bool boolstatus;  
  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
            swDrawing = (DrawingDoc)swModel;  
  
            swModelDocExt = (ModelDocExtension)swModel.Extension;  
            boolstatus = swDrawing.ActivateView("Drawing View2");  
            boolstatus = swModelDocExt.SelectByID2("Drawing View2", "DRAWINGVIEW",  
0.4426278247583, 0.3837831481976, 0, false, 0, null, 0);
```

```

swView = (View)swDrawing.ActiveDrawingView;
typeDrawingView = swView.Type;
if (typeDrawingView == 4)
{
    Debug.Print("Type of selected drawing view is projected.");
}
else
{
    Debug.Print("Type of selected drawing view is not projected. Exiting macro.");
    return;
}

swProjectionArrow = (ProjectionArrow)swView.GetProjectionArrow();
if ((swProjectionArrow != null))
{
    Debug.Print("Projection arrow visible: " + swProjectionArrow.Visible);
}
else
{
    Debug.Print("No projection arrow in projected view.");
}

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;

}
}

```

Is There a Projection Arrow and Is It Visible Example (VBA)

This example shows how to find out if:

- the selected drawing view is a projected view
- a projection arrow exists in the projected view
- the projected arrow is visible

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.  
' 2. Double-click the projected view (i.e., the upper-right drawing view).  
' 3. In the PropertyManager page, click the Arrow check box and  
'    type a label for the projection arrow.  
' 4. Click OK to close the PropertyManager page.  
'  
' Postconditions:  
' 1. Creates a visible projection arrow for Drawing View2, which is  
'    a projected view.  
' 2. Examine the graphics area and Immediate window.  
'  
' NOTE: Because this drawing document is used elsewhere, do not save  
' changes.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swDrawing As SldWorks.DrawingDoc  
Dim swView As SldWorks.View  
Dim swProjectionArrow As SldWorks.ProjectionArrow  
Dim typeDrawingView As Long  
Dim boolstatus As Boolean  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swDrawing = swModel  
  
    Set swModelDocExt = swModel.Extension  
    boolstatus = swDrawing.ActivateView("Drawing View2")  
    boolstatus = swModelDocExt.SelectByID2("Drawing View2", "DRAWINGVIEW", 0.4426278247583,  
    0.3837831481976, 0, False, 0, Nothing, 0)  
  
    Set swView = swDrawing.ActiveDrawingView  
    typeDrawingView = swView.Type  
    If typeDrawingView = 4 Then  
        Debug.Print "Type of selected drawing view is projected."  
    Else  
        Debug.Print "Type of selected drawing view is not projected. Exiting macro."    End If  
End Sub
```

```
        Exit Sub
    End If

    Set swProjectionArrow = swView.GetProjectionArrow
    If Not swProjectionArrow Is Nothing Then
        Debug.Print "Projection arrow visible: " & swProjectionArrow.Visible
    Else
        Debug.Print "No projection arrow in projected view."
    End If
End Sub
```

Is There a Projection Arrow and Is It Visible Example (VB.NET)

This example shows how to find out if:

- the selected drawing view is a projected view
- a projection arrow exists in the projected view
- the projected arrow is visible

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.  
' 2. Double-click the projected view (i.e., the upper-right drawing view).  
' 3. In the PropertyManager page, click the Arrow check box and  
'    type a label for the projection arrow.  
' 4. Click OK to close the PropertyManager page.  
'  
' Postconditions:  
' 1. Creates a visible projection arrow for Drawing View2, which is  
'    a projected view.  
' 2. Examine the graphics area and Immediate window.  
'  
' NOTE: Because this drawing document is used elsewhere, do not save  
' changes.  
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub main()
```

```
        Dim swModel As ModelDoc2
```

```
        Dim swModelDocExt As ModelDocExtension
```

```
        Dim swDrawing As DrawingDoc
```

```
        Dim swView As View
```

```
        Dim swProjectionArrow As ProjectionArrow
```

```
        Dim typeDrawingView As Integer
```

```
        Dim boolstatus As Boolean
```

```
        swModel = swApp.ActiveDoc
```

```
        swDrawing = swModel
```

```
        swModelDocExt = swModel.Extension
```

```
        boolstatus = swDrawing.ActivateView("Drawing View2")
```

```
        boolstatus = swModelDocExt.SelectByID("Drawing View2", "DRAWINGVIEW", 0.4426278247583,  
0.3837831481976, 0, False, 0, Nothing, 0)
```

```
        swView = swDrawing.ActiveDrawingView
```

```
        typeDrawingView = swView.Type
```

```
If typeDrawingView = 4 Then
    Debug.Print("Type of selected drawing view is projected.")
Else
    Debug.Print("Type of selected drawing view is not projected. Exiting macro.")
    Exit Sub
End If
```

```
swProjectionArrow = swView.GetProjectionArrow
If Not swProjectionArrow Is Nothing Then
    Debug.Print("Projection arrow visible: " & swProjectionArrow.Visible)
Else
    Debug.Print("No projection arrow in projected view.")
End If
```

```
End Sub
```

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

```
End Class
```

Isolate Changed Dimension Example (C#)

This example shows how to isolate a changed dimension.

```
//-----  
// Preconditions: The specified drawing and part  
// documents exist.  
//  
// Postconditions:  
// 1. Opens the drawing document.  
// 2. Sets the system option to display  
//    changed dimensions in the color selected  
//    for Tools > Options > System Options >  
//    Colors > Color scheme settings >  
//    Drawings, Changed dimensions.  
// 3. Saves and closes the drawing document.  
// 4. Opens the part document of the drawing document.  
// 5. Changes a dimension.  
// 6. Saves and closes the part document.  
// 7. Opens the previously saved drawing document.  
// 8. Examine the drawing document to verify that  
//    the changed dimension is displayed in the  
//    changed-dimension color. Place your cursor over  
//    the dimension to see its previous value.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System;  
  
namespace IsolateChangedDimensionsDrawingDocCSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        public void Main()  
        {  
            ModelDoc2 swModel = default(ModelDoc2);  
            ModelDocExtension swModelDocExt = default(ModelDocExtension);  
            DrawingDoc swDrawing = default(DrawingDoc);  
            string fileName = null;  
            string saveFileName = null;  
            int errors = 0;  
            int warnings = 0;  
            bool status = false;  
  
            // Open drawing document  
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS  
2018\\samples\\tutorial\\api\\box.slddrw";  
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocDRAWING,  
(int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);  
  
            // Isolate changed dimensions  
            // Equivalent to selecting Tools > Options > System Options > Colors >  
            // Use specified color for changed drawing dimensions on open  
            swApp.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swUseChangedDimensions,  
true);  
  
            swDrawing = (DrawingDoc)swModel;  
            swDrawing.IsolateChangedDimensions();  
  
            // Save drawing document to another name  
            saveFileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS  
2018\\samples\\tutorial\\api\\box_changed.slddrw";  
            swModelDocExt = (ModelDocExtension)swModel.Extension;  
            status = swModelDocExt.SaveAs(saveFileName,  
(int)swSaveAsVersion_e.swSaveAsCurrentVersion, (int)swSaveAsOptions_e.swSaveAsOptions_Silent,
```

```

null, ref errors, ref warnings);
    swApp.CloseDoc(saveFileName);

    // Open the part document referenced by the drawing document,
    // change a dimension, and save the document
    fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\api\\box.sldprt";
    swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART,
(int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
    swModelDocExt = (ModelDocExtension)swModel.Extension;
    status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, true, 0, null, 0);
    status = swModelDocExt.SelectByID2("D2@Sketch1@box.SLDPR", "DIMENSION",
-0.03613329319351, -0.02215939491444, 0.02938582119709, true, 0, null, 0);
    Dimension swDimension = default(Dimension);
    swDimension = (Dimension)swModel.Parameter("D2@Sketch1");
    swDimension.SystemValue = 0.185;
    swModel.ClearSelection2(true);
    status = swModel.EditRebuild3();
    status = swModel.Save3((int)swSaveAsOptions_e.swSaveAsOptions_Silent, ref errors, ref
warnings);
    swApp.CloseDoc(fileName);

    // Open the previously saved drawing document
    // and place your cursor on the changed dimension,
    // which displays in the color specified for
    // changed dimensions, to see its previous value
    swModel = (ModelDoc2)swApp.OpenDoc6(saveFileName,
(int)swDocumentTypes_e.swDocDRAWING, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref
errors, ref warnings);
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>

    public SldWorks swApp;

    }
}

```

Isolate Changed Dimension Example (VBA)

This example shows how to isolate a changed dimension.

```
'-----  
' Preconditions: Verify that the specified drawing and part  
' documents exist.  
'  
' Postconditions:  
' 1. Opens the drawing document.  
' 2. Sets the system option to display  
'    changed dimensions in the color selected  
'    for Tools > Options > System Options >  
'    Colors > Color scheme settings >  
'    Drawings, Changed dimensions.  
' 3. Saves and closes the drawing document.  
' 4. Opens the part document of the drawing document.  
' 5. Changes a dimension.  
' 6. Saves and closes the part document.  
' 7. Opens the previously saved drawing document.  
' 8. Examine the drawing document to verify that  
'    the changed dimension is displayed in the  
'    changed-dimension colors. Place your cursor over  
'    the dimension to see its previous value.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swDrawing As SldWorks.DrawingDoc  
Dim fileName As String  
Dim saveFileName As String  
Dim errors As Long  
Dim warnings As Long  
Dim status As Boolean  
  
Sub main()  
  
Set swApp = Application.SldWorks  
  
' Open drawing document  
fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\box.slddrw"  
Set swModel = swApp.OpenDoc6(fileName, swDocDRAWING, swOpenDocOptions_Silent, "", errors, warnings)  
  
' Isolate changed dimensions  
' Equivalent to selecting Tools > Options > System Options > Colors >  
' Use specified color for changed drawing dimensions on open  
swApp.SetUserPreferenceToggle swUserPreferenceToggle_e.swUseChangedDimensions, True  
Set swDrawing = swModel  
swDrawing.IsolateChangedDimensions  
  
' Save drawing document to another name  
saveFileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\box_changed.slddrw"  
Set swModelDocExt = swModel.Extension  
status = swModelDocExt.SaveAs(saveFileName, swSaveAsCurrentVersion, swSaveAsOptions_Silent, Nothing, errors,  
warnings)  
swApp.CloseDoc (saveFileName)  
  
' Open the part document referenced by the drawing document,  
' change a dimension, and save the document  
fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\box.sldprt"  
Set swModel = swApp.OpenDoc6(fileName, swDocPART, swOpenDocOptions_Silent, "", errors, warnings)  
Set swModelDocExt = swModel.Extension  
status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, True, 0, Nothing, 0)  
status = swModelDocExt.SelectByID2("D2@Sketch1@box.SLDPRT", "DIMENSION", -0.03613329319351, -0.02215939491444,  
0.02938582119709, True, 0, Nothing, 0)  
Dim swDimension As SldWorks.Dimension  
Set swDimension = swModel.Parameter("D2@Sketch1")  
swDimension.SystemValue = 0.185
```

```
swModel.ClearSelection2 True
status = swModel.EditRebuild3()
status = swModel.Save3(swSaveAsOptions_Silent, errors, warnings)
swApp.CloseDoc (fileName)

' Open the previously saved drawing document
' and place your cursor on the changed dimension,
' which displays in the color specified for
' changed dimensions, to see its previous value
Set swModel = swApp.OpenDoc6(saveFileName, swDocDRAWING, swOpenDocOptions_Silent, "", errors, warnings)

End Sub
```

Isolate Changed Dimension Example (VB.NET)

This example shows how to isolate a changed dimension.

```
'-----  
' Preconditions: Verify that the specified drawing and part  
' documents exist.  
'  
' Postconditions:  
' 1. Opens the drawing document.  
' 2. Sets the system option to display  
'    changed dimensions in the color selected  
'    for Tools > Options > System Options >  
'    Colors > Color scheme settings >  
'    Drawings, Changed dimensions.  
' 3. Saves and closes the drawing document.  
' 4. Opens the part document of the drawing document.  
' 5. Changes a dimension.  
' 6. Saves and closes the part document.  
' 7. Opens the previously saved drawing document.  
' 8. Examine the drawing document to verify that  
'    the changed dimension is displayed in the  
'    changed-dimension color. Place your cursor over  
'    the dimension to see its previous value.  
'-----  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System  
  
Partial Class SolidWorksMacro  
  
    Public Sub Main()  
  
        Dim swModel As ModelDoc2  
        Dim swModelDocExt As ModelDocExtension  
        Dim swDrawing As DrawingDoc  
        Dim fileName As String  
        Dim saveFileName As String  
        Dim errors As Long  
        Dim warnings As Long  
        Dim status As Boolean  
  
        ' Open drawing document  
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS  
2018\samples\tutorial\api\box.slddrw"  
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING,  
swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)  
  
        ' Isolate changed dimensions  
        ' Equivalent to selecting Tools > Options > System Options > Colors >  
        ' Use specified color for changed drawing dimensions on open  
        swApp.SetUserPreferenceToggle(swUserPreferenceToggle_e.swUseChangedDimensions, True)  
        swDrawing = swModel  
        swDrawing.IsolateChangedDimensions()  
  
        ' Save drawing document to another name  
        saveFileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS  
2018\samples\tutorial\api\box_changed.slddrw"  
        swModelDocExt = swModel.Extension  
        status = swModelDocExt.SaveAs(saveFileName, swSaveAsVersion_e.swSaveAsCurrentVersion,  
swSaveAsOptions_e.swSaveAsOptions_Silent, Nothing, errors, warnings)  
        swApp.CloseDoc(saveFileName)  
  
        ' Open the part document referenced by the drawing document,
```

```

' change a dimension, and save the document
fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\api\box.sldprt"
swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART,
swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
swModelDocExt = swModel.Extension
status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, True, 0, Nothing, 0)
status = swModelDocExt.SelectByID2("D2@Sketch1@box.SLDPR", "DIMENSION",
-0.03613329319351, -0.02215939491444, 0.02938582119709, True, 0, Nothing, 0)
Dim swDimension As Dimension
swDimension = swModel.Parameter("D2@Sketch1")
swDimension.SystemValue = 0.185
swModel.ClearSelection2(True)
status = swModel.EditRebuild3()
status = swModel.Save3(swSaveAsOptions_e.swSaveAsOptions_Silent, errors, warnings)
swApp.CloseDoc(fileName)

' Open the previously saved drawing document
' and place your cursor on the changed dimension,
' which displays in the color specified for
' changed dimensions, to see its previous value
swModel = swApp.OpenDoc6(saveFileName, swDocumentTypes_e.swDocDRAWING,
swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
End Sub

```

```

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

Isolate Component Example (C#)

This example shows how to isolate a component and save the characteristics of the new display state.

```
//-----
// Preconditions: Verify that the assembly exists.
//
// Postconditions:
// 1. Opens the assembly.
// 2. Selects a component.
// 3. Isolates the selected component and changes the
//    display of all of the other components to wireframe.
// 4. Saves the display characteristics to a new display
//    state.
// 5. Exits isolate.
// 6. Click the ConfigurationManager tab and double-click
//    Test_Display_State.
//
// NOTE: Because the assembly is used elsewhere, do not
// save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            AssemblyDoc swAssembly = default(AssemblyDoc);
            bool status = false;
            int errors = 0;
            int warnings = 0;
            string assemblyName = null;
            string componentToIsolate = null;

            assemblyName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\api\\interferenceassem.sldasm";
            swModel = (ModelDoc2)swApp.OpenDoc6(assemblyName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDoc
Options_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swAssembly = (AssemblyDoc)swModel;
            componentToIsolate = "squarepad_pink-1@interferenceassem";
            status = swModelDocExt.SelectByID2(componentToIsolate, "COMPONENT", 0, 0, 0, false, 0, null, 0);

            //Isolate selected component and set the
            //visibility of all of the other components to wireframe
            status = swAssembly.Isolate();
            swAssembly.SetIsolateVisibility((int)swIsolateVisibility_e.swIsolateVisibility_WIREFRAME);

            //Save the new display state as Test_Display_State
            status = swAssembly.SaveIsolate("Test_Display_State");

            //Exit isolate
            status = swAssembly.ExitIsolate();
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Isolate Component Example (VBA)

This example shows how to isolate a component and save the characteristics of the new display state.

```
'-----  
' Preconditions: Verify that the assembly exists.  
'  
' Postconditions:  
' 1. Opens the assembly.  
' 2. Selects a component.  
' 3. Isolates the selected component and changes the  
'    display of all of the other components to wireframe.  
' 4. Saves the display characteristics to a new display  
'    state.  
' 5. Exits isolate.  
' 6. Click the ConfigurationManager tab and double-click  
'    Test_Display_State.  
'  
' NOTE: Because the assembly is used elsewhere, do not  
'    save changes.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swAssembly As SldWorks.AssemblyDoc  
Dim status As Boolean  
Dim errors As Long  
Dim warnings As Long  
Dim assemblyName As String  
Dim componentToIsolate As String  
  
Sub main()  
    Set swApp = Application.SldWorks  
    assemblyName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS  
2018\samples\tutorial\api\interferenceassem.sldasm"  
    Set swModel = swApp.OpenDoc6(assemblyName, swDocumentTypes_e.swDocASSEMBLY,  
swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)  
    Set swModelDocExt = swModel.Extension  
    Set swAssembly = swModel  
  
    componentToIsolate = "squarepad_pink-1@interferenceassem"  
    status = swModelDocExt.SelectByID2(componentToIsolate, "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)  
  
    'Isolate selected component and set the  
    'visibility of all of the other components to wireframe  
    status = swAssembly.Isolate()  
    swAssembly.SetIsolateVisibility (swIsolateVisibility_e.swIsolateVisibility_WIREFRAME)  
  
    'Save the new display state as Test_Display_State  
    status = swAssembly.SaveIsolate("Test_Display_State")  
  
    'Exit isolate  
    status = swAssembly.ExitIsolate()  
  
End Sub
```

Isolate Component Example (VB.NET)

This example shows how to isolate a component and save the characteristics of the new display state.

```
'-----
' Preconditions: Verify that the assembly exists.
'
' Postconditions:
' 1. Opens the assembly.
' 2. Selects a component.
' 3. Isolates the selected component and changes the
'    display of all of the other components to wireframe.
' 4. Saves the display characteristics to a new display
'    state.
' 5. Exits isolate.
' 6. Click the ConfigurationManager tab and double-click
'    Test_Display_State.
'
' NOTE: Because the assembly is used elsewhere, do not
' save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swAssembly As AssemblyDoc
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim assemblyName As String
        Dim componentToIsolate As String

        assemblyName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\api\interferenceassem.sldasm"
        swModel = swApp.OpenDoc6(assemblyName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOption
s_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension
        swAssembly = swModel
        componentToIsolate = "squarepad_pink-1@interferenceassem"
        status = swModelDocExt.SelectByID2(componentToIsolate, "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)

        'Isolate selected component and set the
        'visibility of all of the other components to wireframe
        status = swAssembly.Isolate()
        swAssembly.SetIsolateVisibility(swIsolateVisibility_e.swIsolateVisibility_WIREFRAME)

        'Save the new display state as Test_Display_State
        status = swAssembly.SaveIsolate("Test_Display_State")

        'Exit isolate
        status = swAssembly.ExitIsolate()

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Iterate Through All Configurations Example (VBA)

This example shows how to iterate through all of the configurations in a document and forcibly rebuild each one.

```
' -----  
' Preconditions:  
' 1. Open a part or assembly.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Iterates through all configurations.  
' 2. Examine the Immediate window.  
' -----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim vConfNameArr As Variant  
    Dim sConfigName As String  
    Dim nStart As Single  
    Dim i As Long  
    Dim bShowConfig As Boolean  
    Dim bRet As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
  
    Debug.Print "File = " + swModel.GetPathName  
  
    vConfNameArr = swModel.GetConfigurationNames  
    For i = 0 To UBound(vConfNameArr)  
        sConfigName = vConfNameArr(i)  
        bShowConfig = swModel.ShowConfiguration2(sConfigName)  
        nStart = Timer  
        bRebuild = swModel.ForceRebuild3(False)  
        Debug.Print "    Configuration = " & sConfigName  
        Debug.Print "    Configuration shown? " & bShowConfig  
        Debug.Print "    Configuration rebuilt? " & bRebuild  
        Debug.Print "    Execution time for this configuration = " & Timer - nStart & " seconds"  
    Next i  
  
End Sub
```

Iterate Through Dimensions in Model Example (VBA)

This example shows how to iterate through the features and dimensions in a part.

```
' -----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\lesson2\tutor1.sldprt.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Iterates through the features and dimensions in the part.  
' 2. Examine the Immediate window.  
'  
' NOTE: Because the part is used elsewhere, do not save changes.  
' -----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swFeat As SldWorks.Feature  
    Dim swSubFeat As SldWorks.Feature  
    Dim swDispDim As SldWorks.DisplayDimension  
    Dim swDim As SldWorks.Dimension  
    Dim swAnn As SldWorks.Annotation  
    Dim bRet As Boolean  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
    Set swFeat = swModel.FirstFeature  
  
    Debug.Print "File = " & swModel.GetPathName  
  
    Do While Not swFeat Is Nothing  
        Debug.Print " " + swFeat.Name  
  
        Set swSubFeat = swFeat.GetFirstSubFeature  
        Do While Not swSubFeat Is Nothing  
            Debug.Print " " + swSubFeat.Name  
            Set swDispDim = swSubFeat.GetFirstDisplayDimension  
            Do While Not swDispDim Is Nothing  
                Set swAnn = swDispDim.GetAnnotation  
                Set swDim = swDispDim.GetDimension  
                Debug.Print " [" & swDim.FullName & "] = " & swDim.GetSystemValue2("")  
                Set swDispDim = swSubFeat.GetNextDisplayDimension(swDispDim)  
            Loop  
            Set swSubFeat = swSubFeat.GetNextSubFeature  
        Loop  
        Set swDispDim = swFeat.GetFirstDisplayDimension  
        Do While Not swDispDim Is Nothing  
            Set swAnn = swDispDim.GetAnnotation  
            Set swDim = swDispDim.GetDimension  
            Debug.Print " [" & swDim.FullName & "] = " & swDim.GetSystemValue2("")  
            Set swDispDim = swFeat.GetNextDisplayDimension(swDispDim)  
        Loop  
        Set swFeat = swFeat.GetNextFeature  
    Loop  
  
End Sub
```

Keep SOLIDWORKS Design Invisible While Activating Documents Example (C#)

This example shows how to keep SOLIDWORKS Design invisible while activating documents, including assembly component files, and saving those documents as PDF files.

```
//-----  
// Preconditions:  
// 1. Verify that the specified assembly file exists.  
// 2. Verify that c:\temp exists.  
// 3. Open the Immediate window.  
//  
// Postconditions:  
// 1. Saves the specified assembly file and its  
//    component files as PDF files to the c:\temp.  
// 2. Examine the Immediate window and c:\temp.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System;  
using System.Diagnostics;  
using System.IO;  
  
namespace Macro1CSharp.csproj  
{  
    public  
    partial class SolidWorksMacro  
    {  
        public  
        ModelDoc2 swModel;  
        public ModelDocExtension swExtension;  
  
        public void Main()  
        {  
            Frame pFrame;  
            string Document = null;  
            string Output = null;  
            try  
            {  
                // Allow SOLIDWORKS Design to run in the background  
                // and be invisible  
                swApp.UserControl = false;  
  
                // If the following property is true, then the  
                // SOLIDWORKS Design frame will be visible on a call to  
                // ISldWorks::ActivateDoc2; so, set it to false  
                swApp.Visible = false;  
  
                // Keep SOLIDWORKS Design frame invisible when  
                // ISldWorks::ActivateDoc2 is called  
                pFrame = (Frame)swApp.Frame();  
                pFrame.KeepInvisible = true;  
  
                Document = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS  
2018\\samples\\tutorial\\advdrawings\\blade shaft.sldasm";  
                Output = "c:\\temp\\";  
                Debug.Print("--- Save files as PDF ---");  
                SaveToPDF(Document, Output);  
                swApp.CloseAllDocuments(true);  
                Debug.Print("--- Done ---");  
  
                // Show SOLIDWORKS Design frame and SOLIDWORKS Design  
                pFrame.KeepInvisible = false;  
                swApp.Visible = true;  
            }  
            catch { }  
        }  
    }  
}
```

```

    }
    catch
    {
        Debug.Print("Execution failed.");
    }
}

```

```

private void SaveToPDF(string docFileName, string outputpath)
{
    AssemblyDoc swAssembly = default(AssemblyDoc);
    int doctype = 0;
    int errors = 0;
    int warnings = 0;
    string modelpath = "";
    string modelFileName = "";
    string convFileName = "";
    bool Success = false;
    object[] vComponents = null;
    long i = 0;

    // Determine the type of SOLIDWORKS Design file based on
    // its filename extension
    string extension = "";
    extension = Path.GetExtension(docFileName);
    if (extension == ".sldprt")
    {
        doctype = (int)swDocumentTypes_e.swDocPART;
    }
    else if (extension == ".SLDPRT")
    {
        doctype = (int)swDocumentTypes_e.swDocPART;
    }
    else if (extension == ".sldasm")
    {
        doctype = (int)swDocumentTypes_e.swDocASSEMBLY;
    }
    else if (extension == ".SLDASM")
    {
        doctype = (int)swDocumentTypes_e.swDocASSEMBLY;
    }
    else if (extension == ".slddrw")
    {
        doctype = (int)swDocumentTypes_e.swDocDRAWING;
    }
    else if (extension == ".SLDDRW")
    {
        doctype = (int)swDocumentTypes_e.swDocDRAWING;
    }
    else
    {
        doctype = (int)swDocumentTypes_e.swDocNONE;
    }

    //Open document
    swModel = (ModelDoc2)swApp.OpenDoc6(docFileName, doctype,
(int)swOpenDocOptions_e.swOpenDocOptions_Silent |
(int)swOpenDocOptions_e.swOpenDocOptions_ReadOnly, "", ref errors, ref warnings);
    if (swModel == null)
    {
        Debug.Print("Failed to open document " + modelpath + ". Errors: " + errors);
    }
    // Activate the document, which should remain invisible
    // due to earlier call to IFrame::KeepInvisible
    swModel = (ModelDoc2)swApp.ActivateDoc2(docFileName, true, ref errors);

    // Build destination filename
    modelpath = swModel.GetPathName();
    modelFileName = Path.GetFileNameWithoutExtension(modelpath);
}

```

```

convFileName = outputpath + modelFileName + ".pdf";
swExtension = (ModelDocExtension)swModel.Extension;

// Save document as PDF
Success = swExtension.SaveAs(convFileName,
(int)swSaveAsVersion_e.swSaveAsCurrentVersion, (int)swSaveAsOptions_e.swSaveAsOptions_Silent,
null, ref errors,
ref warnings);
if (Success)
{
    Debug.Print("Document, " + modelpath + ", saved as " + convFileName + ". ");
}
else
{
    Debug.Print("Document not saved: ");
    Debug.Print(" Errors: " + errors + modelpath + " as " + convFileName + ". ");
}
// Process all components
if (doctype == (int)swDocumentTypes_e.swDocASSEMBLY)
{
    swAssembly = (AssemblyDoc)swModel;
    vComponents = (object[])swAssembly.GetComponents(true);
    for (i = 0; i < vComponents.Length; i++)
    {
        Component2 swComponent = default(Component2);
        swComponent = (Component2)vComponents[i];
        SaveToPDF(swComponent.GetPathName(), outputpath);
    }
}

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Keep SOLIDWORKS Design Invisible While Activating Documents Example (VBA)

This example shows how to keep SOLIDWORKS Design invisible while activating SOLIDWORKS Design documents, including assembly component files, and saving those documents as PDF files.

```
'-----  
' Preconditions:  
' 1. Verify that the specified assembly file exists.  
' 2. Verify that c:\temp exists.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Saves the specified assembly file and its  
'    component files as PDF files to the c:\temp.  
' 2. Examine the Immediate window and c:\temp.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swExtension As SldWorks.ModelDocExtension  
Dim swAssembly As SldWorks.AssemblyDoc  
Dim doctype As Long  
Dim errors As Long  
Dim warnings As Long  
Dim modelpath As String  
Dim modelFileName As String  
Dim convFileName As String  
Dim Success As Boolean  
Dim vComponents As Variant  
Dim i As Long  
Dim outputpath As String  
  
Sub SaveToPDF(docFileName As String, outputpath As String)  
    ' Determine the type of SOLIDWORKS Design file based on  
    ' its filename extension  
    If LCase(Right(docFileName, 7)) = ".sldprt" Then  
        doctype = swDocPART  
    ElseIf LCase(Right(docFileName, 7)) = ".sldasm" Then  
        doctype = swDocASSEMBLY  
    ElseIf LCase(Right(docFileName, 7)) = ".slddrw" Then  
        doctype = swDocDRAWING  
    Else  
        doctype = swDocNONE  
    End If  
  
    ' Open document  
    Set swModel = swApp.OpenDoc6(docFileName, doctype, swOpenDocOptions_Silent Or  
swOpenDocOptions_ReadOnly, "", errors, warnings)  
    If swModel Is Nothing Then  
        Debug.Print "Failed to open document " + modelpath + ". Errors: " & errors  
    End If  
  
    ' Activate the document, which should remain invisible  
    ' due to earlier call to IFrame::KeepInvisible  
    Set swModel = swApp.ActivateDoc2(docFileName, True, errors)  
  
    ' Build destination file name  
    modelpath = swModel.GetPathName()  
    modelFileName = Mid(modelpath, InStrRev(modelpath, "\") + 1)  
    modelFileName = Left(modelFileName, InStrRev(modelFileName, ".") - 1)
```

```

convFileName = outputpath + modelFileName + ".pdf"

' Save document as PDF
Set swExtension = swModel.Extension
Success = swExtension.SaveAs(convFileName, swSaveAsCurrentVersion, swSaveAsOptions_Silent,
Nothing, errors, warnings)

If Success Then
    Debug.Print "Document, " + modelpath + ", saved as " + convFileName + ". "
Else
    Debug.Print "Document not saved: "
    Debug.Print "  Errors: " & errors + modelpath + " as " + convFileName + ". "
End If

' Process all components
If doctype = swDocASSEMBLY Then
    Set swAssembly = swModel
    vComponents = swAssembly.GetComponents(True)
    For i = 0 To UBound(vComponents)
        Dim swComponent As SldWorks.Component2
        Set swComponent = vComponents(i)
        SaveToPDF swComponent.GetPathName(), outputpath
    Next i
End If
End Sub

Sub main()
    On Error GoTo Fail:

    ' Get SOLIDWORKS Design
    Set swApp = Application.SldWorks

    ' Allow SOLIDWORKS Design to run in the background
    ' and be invisible
    swApp.UserControl = False

    ' If the following property is true, then the
    ' SOLIDWORKS Design frame will be visible on a call to
    ' ISldWorks::ActivateDoc2; so set it to false
    swApp.visible = False

    ' Keep SOLIDWORKS Design frame invisible when
    ' ISldWorks::ActivateDoc2 is called
    Dim pFrame As Frame
    Set pFrame = swApp.Frame
    pFrame.KeepInvisible = True

    Dim Document As String
    Dim Output As String
    Document = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\advdrawings\blade shaft.sldasm"
    Output = "c:\temp\"

    Debug.Print "--- Save files as PDF ---"
    SaveToPDF Document, Output
    swApp.CloseAllDocuments True
    Debug.Print "--- Done ---"

    ' Show SOLIDWORKS Design frame and SOLIDWORKS Design
    pFrame.KeepInvisible = False
    swApp.visible = True
    Exit Sub

Fail:
    Debug.Print "Execution failed with error " & Err.Number & ": '" & Err.Description & "'"

```

End Sub

Keep SOLIDWORKS Design Invisible While Activating Documents Example (VB.NET)

This example shows how to keep SOLIDWORKS Design invisible while activating SOLIDWORKS Design documents, including assembly component files, and saving those documents as PDF files.

```
'-----
' Preconditions:
' 1. Verify that the specified assembly file exists.
' 2. Verify that c:\temp exists.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Saves the specified assembly file and its
'    component files as PDF files to the c:\temp.
' 2. Examine the Immediate window and c:\temp.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public swModel As ModelDoc2
    Public swExtension As ModelDocExtension

    Public Sub main()

        Dim pFrame As Frame
        Dim Document As String
        Dim Output As String

        On Error GoTo Fail

        ' Allow SOLIDWORKS Design to run in the background
        ' and be invisible
        swApp.UserControl = False

        ' If the following property is true, then the
        ' SOLIDWORKS Design frame will be visible on a call to
        ' ISldWorks::ActivateDoc2; so, set it to false
        swApp.Visible = False

        ' Keep SOLIDWORKS Design frame invisible when
        ' ISldWorks::ActivateDoc2 is called
        pFrame = swApp.Frame
        pFrame.KeepInvisible = True

        Document = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\advdrawings\blade shaft.sldasm"
        Output = "c:\temp\"

        Debug.Print("--- Save files as PDF ---")
        SaveToPDF(Document, Output)
        swApp.CloseAllDocuments(True)
        Debug.Print("--- Done ---")

        ' Show SOLIDWORKS Design frame and SOLIDWORKS Design
        pFrame.KeepInvisible = False
```

```

        swApp.Visible = True
    Exit Sub
Fail:
    Debug.Print("Execution failed with error " & Err.Number & ": '" & Err.Description & "'")

End Sub

Private Sub SaveToPDF(ByVal docFileName As String, ByVal outputpath As String)
    Dim swAssembly As AssemblyDoc
    Dim doctype As Integer
    Dim errors As Integer
    Dim warnings As Integer
    Dim modelpath As String = ""
    Dim modelFileName As String = ""
    Dim convFileName As String = ""
    Dim Success As Boolean
    Dim vComponents As Object
    Dim i As Integer

    ' Determine the type of SOLIDWORKS Design file based on
    ' its filename extension
    If LCase(Right(docFileName, 7)) = ".sldprt" Then
        doctype = swDocumentTypes_e.swDocPART
    ElseIf LCase(Right(docFileName, 7)) = ".sldasm" Then
        doctype = swDocumentTypes_e.swDocASSEMBLY
    ElseIf LCase(Right(docFileName, 7)) = ".slddrw" Then
        doctype = swDocumentTypes_e.swDocDRAWING
    Else
        doctype = swDocumentTypes_e.swDocNONE
    End If

    ' Open document
    swModel = swApp.OpenDoc6(docFileName, doctype, swOpenDocOptions_e.swOpenDocOptions_Silent
Or swOpenDocOptions_e.swOpenDocOptions_ReadOnly, "", errors, warnings)
    If swModel Is Nothing Then
        Debug.Print("Failed to open document " + modelpath + ". Errors: " & errors)
    End If

    ' Activate the document, which should remain invisible
    ' due to earlier call to IFrame::KeepInvisible
    swModel = swApp.ActivateDoc2(docFileName, True, errors)

    ' Build destination filename
    modelpath = swModel.GetPathName()
    modelFileName = Mid(modelpath, InStrRev(modelpath, "\") + 1)
    modelFileName = Left(modelFileName, InStrRev(modelFileName, ".") - 1)
    convFileName = outputpath + modelFileName + ".pdf"

    ' Save document as PDF
    swExtension = swModel.Extension
    Success = swExtension.SaveAs(convFileName, swSaveAsVersion_e.swSaveAsCurrentVersion,
swSaveAsOptions_e.swSaveAsOptions_Silent, Nothing, errors, warnings)
    If Success Then
        Debug.Print("Document, " + modelpath + ", saved as " + convFileName + ". ")
    Else
        Debug.Print("Document not saved: ")
        Debug.Print(" Errors: " & errors + modelpath + " as " + convFileName + ". ")
    End If

    ' Process all components
    If doctype = swDocumentTypes_e.swDocASSEMBLY Then
        swAssembly = swModel
        vComponents = swAssembly.GetComponents(True)
        For i = 0 To UBound(vComponents)
            Dim swComponent As Component2
            swComponent = vComponents(i)
            SaveToPDF(swComponent.GetPathName(), outputpath)
        Next i
    End If
End Sub

```

```
        Next i  
    End If
```

```
End Sub
```

```
''' <summary>  
''' The SldWorks swApp variable is pre-assigned for you.  
''' </summary>  
Public swApp As SldWorks
```

```
End Class
```

Link Display States to Configurations Example (C#)

This example shows how to link and unlink display states to and from configurations.

```
// -----  
// Preconditions:  
// 1. Open public_documents\samples\tutorial\pdmworks\speaker.sldasm  
// 2. Open the Immediate window.  
//  
// Postconditions:  
// 1. Gets and sets whether display states are linked to the  
//    active configuration.  
// 2. Closes the assembly document without saving  
//    any changes.  
//-----  
using Microsoft.VisualBasic;  
using System;  
using System.Collections;  
using System.Collections.Generic;  
using System.Data;  
using System.Diagnostics;  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
  
namespace LinkDisplayStatesToConfigurationCSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        public void Main()  
        {  
            ModelDoc2 swModel = default(ModelDoc2);  
            ConfigurationManager swConfigMgr = default(ConfigurationManager);  
            string assemblyName = null;  
  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
            Debug.Print("");  
  
            swConfigMgr = (ConfigurationManager)swModel.ConfigurationManager;  
            swConfigMgr.LinkDisplayStatesToConfigurations = false;  
            Debug.Print("Are display states linked to configurations? " +  
swConfigMgr.LinkDisplayStatesToConfigurations);  
            if (!swConfigMgr.LinkDisplayStatesToConfigurations)  
            {  
                Debug.Print("All display states are available to the active configuration.");  
            }  
  
            swConfigMgr.LinkDisplayStatesToConfigurations = true;  
            Debug.Print("Are display states linked configurations? " +  
swConfigMgr.LinkDisplayStatesToConfigurations);  
            if (swConfigMgr.LinkDisplayStatesToConfigurations)  
            {  
                Debug.Print("All display states are not available to the active configuration.");  
            }  
  
            assemblyName = swModel.GetTitle();  
            swApp.QuitDoc(assemblyName);  
        }  
  
        /// <summary>  
        /// The SldWorks swApp variable is pre-assigned for you.  
        /// </summary>  
  
        public SldWorks swApp;
```


Link Display States to Configurations Example (VBA)

This example shows how to link and unlink display states to and from configurations.

```
'-----  
' Preconditions:  
' 1. Verify that the specified assembly document  
'    to open exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens the specified assembly document.  
' 2. Follow the instructions in the macro at  
'    each Stop statement, which gets and sets whether  
'    display states are linked to the active configuration.  
' 3. Closes the assembly document without saving  
'    any changes.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swConfigMgr As SldWorks.ConfigurationManager  
Dim assemblyFile As String  
Dim errors As Long  
Dim warnings As Long  
Dim assemblyName As String  
  
Sub main()  
    ' Open assembly document  
    assemblyFile = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS  
2018\samples\tutorial\pdmworks\speaker.sldasm"  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.OpenDoc6(assemblyFile, swDocASSEMBLY, swOpenDocOptions_Silent, "", errors, warnings)  
    Stop  
    ' 1. Click the ConfigurationManager tab in the Manager Pane  
    '    in the assembly document.  
    ' 2. Right-click the active display state and click Properties.  
    ' 3. Verify that the "Link display states to configurations" check box  
    '    is selected and click OK.  
    ' 4. Click the Continue button in the IDE.  
  
    Set swConfigMgr = swModel.ConfigurationManager  
    swConfigMgr.LinkDisplayStatesToConfigurations = False  
    Debug.Print "Are display states linked to configurations? " & swConfigMgr.LinkDisplayStatesToConfigurations  
    If Not swConfigMgr.LinkDisplayStatesToConfigurations Then  
        Debug.Print "All display states are available to the active configuration."  
    End If  
    Debug.Print ""  
    Stop  
    ' 1. Click the ConfigurationManager tab again.  
    ' 2. Right-click the active display state and click Properties.  
    ' 3. Verify that the "Link display states to configurations" check box  
    '    is not selected and click OK.  
    ' 4. Click the Continue button in the IDE.  
  
    swConfigMgr.LinkDisplayStatesToConfigurations = True  
    Debug.Print "Are display states linked configurations? " & swConfigMgr.LinkDisplayStatesToConfigurations  
    If swConfigMgr.LinkDisplayStatesToConfigurations Then  
        Debug.Print "All display states are not available to the active configuration."  
    End If  
    Stop  
    ' 1. Click the ConfigurationManager tab again.  
    ' 2. Right-click the active display state and click Properties.  
    ' 3. Verify that the "Link display states to configurations" check box  
    '    is selected and click OK.
```

' 4. Click the Continue button in the IDE.

```
assemblyName = swModel.GetTitle  
swApp.QuitDoc assemblyName
```

End Sub

Link Display States to Configurations Example(VB.NET)

This example shows how to link and unlink display states to and from configurations.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\pdmworks\speaker.sldasm.  
' 2. Open the Immediate window.  
'-----  
' Postconditions:  
' 1. Gets and sets whether display states are linked to  
'    the active configuration.  
' 2. Closes the assembly document without saving  
'    any changes.  
'-----  
  
Imports SolidWorks.Interop.sldworks  
  
Imports SolidWorks.Interop.swconst  
  
Imports System.Runtime.InteropServices  
  
Imports System.Diagnostics  
  
Imports System  
  
Partial Class SolidWorksMacro  
  
    Public Sub Main()  
  
        Dim swModel As ModelDoc2  
  
        Dim swConfigMgr As ConfigurationManager  
  
        Dim assemblyName As String  
  
        swModel = swApp.ActiveDoc  
  
        Debug.Print("")  
  
        swConfigMgr = swModel.ConfigurationManager  
  
        swConfigMgr.LinkDisplayStatesToConfigurations = False  
  
        Debug.Print("Are display states linked to configurations? " &  
swConfigMgr.LinkDisplayStatesToConfigurations)  
  
        If Not swConfigMgr.LinkDisplayStatesToConfigurations Then  
            Debug.Print("All display states are available to the active configuration.")  
        End If  
  
        swConfigMgr.LinkDisplayStatesToConfigurations = True  
  
        Debug.Print("Are display states linked configurations? " &  
swConfigMgr.LinkDisplayStatesToConfigurations)  
  
        If swConfigMgr.LinkDisplayStatesToConfigurations Then  
            Debug.Print("All display states are not available to the active configuration.")  
        End If  
    End Sub  
End Class
```

End If

```
assemblyName = swModel.GetTitle  
swApp.QuitDoc(assemblyName)
```

End Sub

```
''' <summary>  
'''  
The SldWorks swApp variable is pre-assigned for you.  
''' </summary>  
  
Public swApp As SldWorks
```

End Class

Link Projected View to Parent Configuration Example (C#)

This example shows how to link a projected drawing view to the parent configuration.

```
//-----  
// Preconditions:  
// 1. Verify that the specified file exists.  
// 2. Open the Immediate window.  
//  
// Postconditions:  
// 1. Links the projected view to the parent configuration.  
// 2. Examine the Immediate window.  
//  
// NOTE: Because this drawing document is used elsewhere, do not save  
// changes.  
//-----  
  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System;  
using System.Diagnostics;  
  
namespace LinkParentConfigurationCSharp.csproj  
{  
  
    partial class SolidWorksMacro  
    {  
  
        public ModelDoc2 swModel;  
        public View swView;  
        public bool status;  
  
        public void Main()  
        {  
  
            ModelDocExtension swModelDocExt = default(ModelDocExtension);  
            DrawingDoc swDrawing = default(DrawingDoc);  
            string fileName = null;  
            int errors = 0;  
            int warnings =  
0;  
  
            // Open drawing document that has a  
projected view  
  
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS  
2018\\samples\\tutorial\\advdrawings\\foodprocessor.slddrw";  
        }  
    }  
}
```

```

        swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocDRAWING,
(int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);

        swDrawing = (DrawingDoc)swModel;

        swModelDocExt = (ModelDocExtension)swModel.Extension;

        // Make the projected view the active
view
        status = swDrawing.ActivateView("Drawing View2");

        status = swModelDocExt.SelectByID2("Drawing
View2", "DRAWINGVIEW",
0.4426278247583, 0.3837831481976, 0, false,
0, null, 0);

        swView = (View)swDrawing.ActiveDrawingView;

        // Determine whether the projected view
is linked to the
        // parent
configuration
        LinkConfiguration();

        // Link the projected view to the parent
        //
configuration
        swView.LinkParentConfiguration = true;

        // Determine whether the projected view
is linked to the
        // parent
configuration
        LinkConfiguration();
    }

    public void LinkConfiguration()
    {
        // Print to the Immediate window whether
        // the
projected view is linked to the parent
        //
configuration
        status = swView.LinkParentConfiguration;

        if (status)
        {
            Debug.Print("Projected view now linked to parent configuration.");

```

```
        swModel.EditRebuild3();  
    }  
    else  
    {  
        Debug.Print("Projected view not linked to parent configuration.");  
    }  
}
```

```
/// <summary>  
/// The SldWorks swApp variable is pre-assigned for you.  
/// </summary>  
public SldWorks swApp;
```

```
}
```

```
}
```

Link Projected View to Parent Configuration Example (VBA)

This example shows how to link a projected drawing view to the parent configuration.

```
'-----
' Preconditions:
' 1. Verify that the specified file exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Links the projected view to the parent configuration.
' 2. Examine the Immediate window.
'
' NOTE: Because this drawing document is used elsewhere, do not save
' changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swDrawing As SldWorks.DrawingDoc
Dim swView As SldWorks.View
Dim fileName As String
Dim errors As Long
Dim warnings As Long
Dim status As Boolean

Sub main()

    Set swApp = Application.SldWorks

    ' Open drawing document that has a projected view
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
    2018\samples\tutorial\advdrawings\foodprocessor.slddrw"
    Set swModel = swApp.OpenDoc6(fileName, swDocDRAWING, swOpenDocOptions_Silent, "", errors, warnings)
    Set swDrawing = swModel
    Set swModelDocExt = swModel.Extension

    ' Make the projected view the active view
    status = swDrawing.ActivateView("Drawing View2")
    status = swModelDocExt.SelectByID2("Drawing View2", "DRAWINGVIEW", 0.4426278247583, 0.3837831481976,
    0, False, 0, Nothing, 0)
    Set swView = swDrawing.ActiveDrawingView

    ' Determine whether the projected view is linked to the
    ' parent configuration
    Call LinkConfiguration

    ' Link the projected view to the parent
    ' configuration
    swView.LinkParentConfiguration = True

    ' Determine whether the projected view is linked to the
    ' parent configuration
    Call LinkConfiguration

End Sub

Private Sub LinkConfiguration()
    ' Print to the Immediate window whether
    ' the projected view is linked to the parent
    ' configuration
    status = swView.LinkParentConfiguration
    If status Then
        Debug.Print "Projected view now linked to parent configuration."
        swModel.EditRebuild3
    Else
        Debug.Print "Projected view not linked to parent configuration."
```

```
End If
End Sub
```

Link Projected View to Parent Configuration Example (VB.NET)

This example shows how to link a projected drawing view to the parent configuration.

```
'-----
' Preconditions:
' 1. Verify that the specified file exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Links the projected view to the parent configuration.
' 2. Examine the Immediate window.
'
' NOTE: Because this drawing document is used elsewhere, do not save
' changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public swModel As ModelDoc2
    Public swView As View
    Public status As Boolean

    Public Sub Main()

        Dim swModelDocExt As ModelDocExtension
        Dim swDrawing As DrawingDoc
        Dim fileName As String
        Dim errors As Integer
        Dim warnings As Integer

        ' Open drawing document that has a projected view
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\advdrawings\foodprocessor.slddrw"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING,
swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swDrawing = swModel
        swModelDocExt = swModel.Extension

        ' Make the projected view the active view
        status = swDrawing.ActivateView("Drawing View2")
        status = swModelDocExt.SelectByID2("Drawing View2", "DRAWINGVIEW", 0.4426278247583,
0.3837831481976, 0, False, 0, Nothing, 0)
        swView = swDrawing.ActiveDrawingView

        ' Determine whether the projected view is linked to the
        ' parent configuration
        Call LinkConfiguration()

        ' Link the projected view to the parent
        ' configuration
        swView.LinkParentConfiguration = True

        ' Determine whether the projected view is linked to the
        ' parent configuration
        Call LinkConfiguration()

    End Sub

    Public Sub LinkConfiguration()
```

```
' Print to the Immediate window whether
' the projected view is linked to the parent
' configuration
status = swView.LinkParentConfiguration
If status Then
    Debug.Print("Projected view now linked to parent configuration.")
    swModel.EditRebuild3()
Else
    Debug.Print("Projected view not linked to parent configuration.")
End If
End Sub
```

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

```
End Class
```

List_Center_Marks_in_Drawing_Example_VB (Topic)

SOLIDWORKS API Help

List Center Marks in Drawing Example (VBA)

This example shows how to list all of the center marks in all of the drawing views in a drawing.

' Preconditions:

- ' 1. Open a drawing with one or more center marks in any view.
- ' 2. Open the Immediate window.

' Postconditions: Examine the Immediate window.

Option Explicit

Public Enum swCenterMarkStyle_e

swCenterMark_NonAnnotation = 1

swCenterMark_Single = 2

swCenterMark_LinearGroup = 3

swCenterMark_CircularGroup = 4

End Enum

Sub main()

Dim swApp As SldWorks.SldWorks

Dim swModel As SldWorks.ModelDoc2

Dim swDraw As SldWorks.DrawingDoc

Dim swView As SldWorks.View

Dim vCenterMarkArr As Variant

Dim vCenterMark As Variant

Dim swCenterMark As SldWorks.CenterMark

Dim bRet As Boolean

Set swApp = CreateObject("SldWorks.Application")

Set swModel = swApp.ActiveDoc

Set swDraw = swModel

Set swView = swDraw.GetFirstView

Debug.Print "File = " & swModel.GetPathName

Do While Not swView Is Nothing

Debug.Print " View = " + swView.GetName2

Set swCenterMark = swView.GetFirstCenterMark2

Do While Not swCenterMark Is Nothing

vCenterMarkArr = swView.GetCentermarks

Debug.Assert Not IsEmpty(vCenterMarkArr)

Debug.Print " CenterLineFont = " & swCenterMark.CenterLineFont

Debug.Print " ConnectionLines = " & swCenterMark.ConnectionLines

' 1 radian = 180°/p = 57.295779513° or approximately 57.3°

Debug.Print " RotationAngle = " & swCenterMark.RotationAngle * 57.3 & " degrees"

Debug.Print " ShowLines = " & swCenterMark.ShowLines

Debug.Print " Size = " & swCenterMark.Size

```
Debug.Print "  Style          = " & swCenterMark.Style  
Debug.Print "  UseDocDisplaySettings  = " & swCenterMark.UseDocDisplaySettings  
Debug.Print ""
```

```
Set swCenterMark = swCenterMark.GetNext
```

```
Loop
```

```
Set swView = swView.GetNextView
```

```
Loop
```

```
End Sub
```

List_Custom_Properties_Example_VB (Topic)

SOLIDWORKS API Help

List Custom Properties Example (VBA)

This example shows how to list the custom properties for a configuration.

```
' -----  
' Preconditions:  
' 1. Open a model document to which custom properties  
'    are assigned.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Iterates through the configurations and  
'    prints configuration-specific and nonconfiguration-  
'    specific custom property values to the  
'    Immediate window.  
' 2. Examine the Immediate window.  
' -----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim vConfigNameArr As Variant  
    Dim vConfigName As Variant  
    Dim vCustInfoNameArr As Variant  
    Dim vCustInfoName As Variant  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
  
    Debug.Print "File = " & swModel.GetPathName  
  
    vConfigNameArr = swModel.GetConfigurationNames  
  
    ' Is empty if a drawing because configurations not supported in drawings  
    If IsEmpty(vConfigNameArr) Then  
        ReDim vConfigNameArr(0)  
        vConfigNameArr(0) = ""  
    Else  
        ' Add a blank string for the nonconfiguration-specific custom properties  
        ReDim Preserve vConfigNameArr(UBound(vConfigNameArr) + 1)  
    End If  
  
    For Each vConfigName In vConfigNameArr  
        Debug.Print vConfigName  
        vCustInfoNameArr = swModel.GetCustomInfoNames2(vConfigName)  
        If Not IsEmpty(vCustInfoNameArr) Then  
            For Each vCustInfoName In vCustInfoNameArr  
                Debug.Print "    " & vCustInfoName  
                Debug.Print "        Type = " & swModel.GetCustomInfoType3(vConfigName, vCustInfoName)  
                Debug.Print "        Value = " & swModel.GetCustomInfoValue(vConfigName, vCustInfoName)  
                Debug.Print "        Text = " & swModel.CustomInfo2(vConfigName, vCustInfoName)  
            Next  
        End If  
        Debug.Print "    -----"  
    Next  
  
End Sub
```

List_End-Cap_Feature_Properties_Example_VB (Topic)

SOLIDWORKS API Help

List End-Cap Feature Properties Example (VBA)

This example shows how to list an end-cap feature's properties and select and then deselect the face used to create the end-cap feature.

```
'-----  
'  
' Preconditions: Model document is open and an end-cap feature is selected.  
'  
' Postconditions: None  
'  
'-----  
  
Option Explicit  
Public Enum swSketchSegments_e  
    swSketchLINE = 0  
    swSketchARC = 1  
    swSketchELLIPSE = 2  
    swSketchSPLINE = 3  
    swSketchTEXT = 4  
    swSketchPARABOLA = 5  
End Enum  
  
Public Enum swSolidworksWeldmentEndCondOptions_e  
    swEndConditionNone = 0  
    swEndConditionMiter = 1  
    swEndConditionButt1 = 2  
    swEndConditionButt2 = 3  
    swEndConditionTrim = 4  
    swEndConditionUseDefault = 5  
End Enum  
  
Sub main()  
    Dim swApp          As SldWorks.SldWorks  
    Dim swModel         As SldWorks.ModelDoc2  
    Dim swSelMgr        As SldWorks.SelectionMgr  
    Dim swFeat          As SldWorks.feature  
    Dim swEndCap        As SldWorks.EndCapFeatureData  
    Dim swFace          As SldWorks.face2  
    Dim swEnt           As SldWorks.Entity  
    Dim i               As Long  
    Dim bRet            As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swFeat = swSelMgr.GetSelectedObject5(1)  
    Set swEndCap = swFeat.GetDefinition  
  
    Debug.Print "File = " & swModel.GetPathName  
    Debug.Print " " & swFeat.Name  
    Debug.Print "   ChamferDistance          = " & swEndCap.ChamferDistance * 1000# & " mm"  
    Debug.Print "   OffsetDistance           = " & swEndCap.OffsetDistance * 1000# & " mm"
```

```
Debug.Print " Thickness = " & swEndCap.Thickness * 1000# & " mm"
Debug.Print " ThicknessRatioForOffset = " & swEndCap.ThicknessRatioForOffset
Debug.Print " UseChamferCorners = " & swEndCap.UseChamferCorners
Debug.Print " UseThicknessRatioForOffset = " & swEndCap.UseThicknessRatioForOffset
```

```
bRet = swEndCap.AccessSelections(swModel, Nothing): Debug.Assert bRet
```

```
Set swFace = swEndCap.Face
```

```
Set swEnt = swFace
```

```
bRet = swEnt.Select4(False, Nothing): Debug.Assert bRet
```

```
Stop
```

```
swEndCap.ReleaseSelectionAccess
```

```
End Sub
```

Load_and_Unload_Add-in_Example_CSharp (Topic)

SOLIDWORKS API Help

Load and Unload Add-in Example (C#)

This example shows how to prompt the user to load or unload an add-in.

```
//-----  
// Preconditions:  
// 1. Create a C# add-in in Microsoft Visual Studio  
//    2010, or later, in C:\test, following the instructions in  
//    Add Shortcut Menu to Add-ins Example \(C#\).  
// 2. Open the Immediate window.  
//  
// Postconditions:  
// 1. Click Yes to load SwCSharpAddin1 or click No to unload  
//    SwCSharpAddin1.  
// 2. Examine the Immediate window.  
//-----  
  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
using System.Windows.Forms;  
using System.Diagnostics;  
  
namespace LoadUnloadAddinsCSharp.csproj  
{  
    public partial class SolidWorksMacro  
    {  
        public void Main()  
        {  
            //Path to add-in  
            const string sAddinName = "C:\\test\\SwCSharpAddin1\\SwCSharpAddin1\\bin\\Debug\\SwCSharpAddin1.dll";  
            int status = 0;  
  
            if (MessageBox.Show("Load add-in?", "Add-  
ins load/unload", MessageBoxButtons.YesNo) == System.Windows.Forms.DialogResult.Yes)  
            {  
                Debug.Print("Loading: " + sAddinName);  
                status = swApp.LoadAddIn(sAddinName);  
            }  
            else  
            {  
                Debug.Print("Unloading: " + sAddinName);  
                status = swApp.UnloadAddIn(sAddinName);  
            }  
  
            Debug.Print(" Status = " + status);  
        }  
  
        /// <summary>  
        /// The SldWorks swApp variable is pre-assigned for you.  
        /// </summary>  
        public SldWorks swApp;  
    }  
}
```

Load and Unload Add-in Example (VBA)

This example shows how to prompt the user to load or unload an add-in.

```
'-----  
' Preconditions:  
' 1. Create a VB.NET add-in in Microsoft Visual Studio  
'    2010, or later, in C:\test, following the instructions in  
'    Add Shortcut Menu to Add-ins Example \(VB.NET\).  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Click Yes to load SwVBAddin1 or click No to unload  
'    swVBAddin1.  
' 2. Examine the Immediate window.  
'-----
```

Option Explicit

Sub main()

```
    'Path to add-in  
    Const sAddinName As String = "C:\test\SwVBAddin1\SwVBAddin1\bin\SwVBAddin1.dll"  
  
    Dim swApp As SldWorks.SldWorks  
    Dim response As Long  
    Dim status As Long  
  
    Set swApp = CreateObject("SldWorks.Application")  
    response = MsgBox("Load add-in?", vbYesNo)  
  
    If response = vbYes Then  
        Debug.Print "Loading: " + sAddinName  
        status = swApp.LoadAddIn(sAddinName)  
    Else  
        Debug.Print "Unloading: " + sAddinName  
        status = swApp.UnloadAddIn(sAddinName)  
    End If  
  
    Debug.Print "  Status = " & status
```

End Sub

Load_and_Unload_Add-in_Example_VBNET (Topic)

SOLIDWORKS API Help

Load and Unload Add-in Example (VB.NET)

This example shows how to prompt the user to load or unload an add-in.

```
'-----
' Preconditions:
' 1. Create a VB.NET add-in in Microsoft Visual Studio
'    2010, or later, in C:\test, following the instructions in
'    Add Shortcut Menu to Add-ins Example \(VB.NET\).
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Click Yes to load SwVBAddin1 or click No to unload
'    SwVBAddin1.
' 2. Examine the Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        'Path to add-in
        Const sAddinName As String = "C:\test\SwVBAddin1\SwVBAddin1\bin\SwVBAddin1.dll"
        Dim response As Integer
        Dim status As Integer

        swApp = CreateObject("SldWorks.Application")
        response = MsgBox("Load add-in?", vbYesNo)
        If response = vbYes Then
            Debug.Print("Loading: " + sAddinName)
            status = swApp.LoadAddIn(sAddinName)
        Else
            Debug.Print("Unloading: " + sAddinName)
            status = swApp.UnloadAddIn(sAddinName)
        End If

        Debug.Print("  Status = " & status)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Locate_Apex_of_Conical_Face_Example_VB (Topic)

SOLIDWORKS API Help

Locate Apex of Conical Face Example (VBA)

This example shows how to locate the apex of a conical face.

```
'-----  
' Preconditions:  
' 1. Open a part or assembly containing a conical face.  
' 2. Select the conical face.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Creates a 3D sketch with a single point at the apex of  
'   the conical face.  
' 2. Examine the Immediate window.  
'  
' NOTE: Conical face can be truncated.  
'-----
```

Option Explicit

Sub main()

```
Dim swApp          As SldWorks.SldWorks  
Dim swMathUtil     As SldWorks.MathUtility  
Dim swModel        As SldWorks.ModelDoc2  
Dim swSelMgr       As SldWorks.SelectionMgr  
Dim swFace         As SldWorks.face2  
Dim swSurf         As SldWorks.surface  
Dim vCone          As Variant  
Dim nAxis(2)       As Double  
Dim vAxis          As Variant  
Dim swAxis         As SldWorks.MathVector  
Dim nOrigin(2)     As Double  
Dim vOrigin        As Variant  
Dim swOrigin       As SldWorks.MathPoint  
Dim swApex         As SldWorks.MathPoint  
Dim swApexPt       As SldWorks.SketchPoint
```

```
Set swApp = Application.SldWorks  
Set swMathUtil = swApp.GetMathUtility  
Set swModel = swApp.ActiveDoc  
Set swSelMgr = swModel.SelectionManager  
Set swFace = swSelMgr.GetSelectedObject5(1)  
Set swSurf = swFace.GetSurface: Debug.Assert swSurf.IsCone
```

```
vCone = swSurf.ConeParams
```

```
nOrigin(0) = vCone(0): nOrigin(1) = vCone(1): nOrigin(2) = vCone(2)  
vOrigin = nOrigin  
Set swOrigin = swMathUtil.CreatePoint((vOrigin))
```

```
nAxis(0) = vCone(3): nAxis(1) = vCone(4): nAxis(2) = vCone(5)  
vAxis = nAxis  
Set swAxis = swMathUtil.CreateVector((vAxis))
```

```
Set swAxis = swAxis.Scale(vCone(6) / Tan(vCone(7)))
```

```
Set swApex = swOrigin.AddVector(swAxis)
```

```
Debug.Print "File = " & swModel.GetPathName
```

```
Debug.Print " origin      = (" & vCone(0) * 1000# & ", " & vCone(1) * 1000# & ", " & vCone(2) * 1000# & ") mm"
```

```
Debug.Print " axis       = (" & vCone(3) & ", " & vCone(4) & ", " & vCone(5) & ")"
```

```
Debug.Print " radius     = " & vCone(6) * 1000# & " mm"
```

```
' 1 radian = 180°/p = 57.295779513° or approximately 57.3°
```

```
Debug.Print " half angle  = " & vCone(7) * 57.3 & " deg"
```

```
Debug.Print " Apex       = " & swApex.ArrayData(0) * 1000# & ", " & swApex.ArrayData(1) * 1000# & ", " & swApex.ArrayData(2) * 1000# & ") mm"
```

```
swModel.Insert3DSketch2 False
```

```
swModel.SetAddToDB True
```

```
Set swApexPt = swModel.CreatePoint2(swApex.ArrayData(0), swApex.ArrayData(1), swApex.ArrayData(2))
```

```
swModel.SetAddToDB False
```

```
swModel.Insert3DSketch2 True
```

```
End Sub
```

Lock All External References Example (VBA)

This example shows how to lock all external references in a part or assembly in reverse chronological order.

```
' -----  
' Preconditions:  
' 1. Open a part or assembly.  
' 1. Open the Immediate window.  
'  
' Postconditions:  
' 1. Locks all external references.  
' 2. Examine the Immediate window.  
' -----  
Option Explicit  
  
Sub ProcessModel(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, sPadStr As String)  
    Debug.Print sPadStr & " Locking: " & swModel.GetPathName  
    swModel.LockAllExternalReferences  
End Sub  
  
Sub ProcessComponent(swApp As SldWorks.SldWorks, swComp As SldWorks.Component2, sPadStr As String)  
    Dim vChildCompArr As Variant  
    Dim vChildComp As Variant  
    Dim swChildComp As SldWorks.Component2  
    Dim swChildModel As SldWorks.ModelDoc2  
  
    vChildCompArr = swComp.GetChildren  
    For Each vChildComp In vChildCompArr  
        Set swChildComp = vChildComp  
        Debug.Print sPadStr & swChildComp.Name2  
        ProcessComponent swApp, swChildComp, sPadStr & " "  
        Set swChildModel = swChildComp.GetModelDoc  
        ProcessModel swApp, swChildModel, sPadStr  
    Next vChildComp  
End Sub  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swAssy As SldWorks.AssemblyDoc  
    Dim swConfigMgr As SldWorks.ConfigurationManager  
    Dim swConf As SldWorks.Configuration  
    Dim swRootComp As SldWorks.Component2  
    Dim nStatus As Long  
    Dim bRet As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swConfigMgr = swModel.ConfigurationManager  
    Set swConf = swConfigMgr.ActiveConfiguration  
    Set swRootComp = swConf.GetRootComponent  
  
    Debug.Print "File = " & swModel.GetPathName  
  
    Select Case swModel.GetType  
        Case swDocPART  
            ProcessModel swApp, swModel, " "  
        Case swDocASSEMBLY  
            Set swAssy = swModel  
            nStatus = swAssy.ResolveAllLightWeightComponents(False)  
            Set swRootComp = swConf.GetRootComponent  
            ProcessComponent swApp, swRootComp, " "  
        Case Else  
            Exit Sub  
    End Select  
End Sub
```

Make All Assembly Components Visible Example (VBA)

This example shows how to make all assembly components visible.

```
' -----  
' Preconditions:  
' 1. Open an assembly.  
' 2. Hide a component (right-click the component in  
'    the FeatureManager design tree and click the  
'    Hide Components button.  
' 3. Open the Immediate window.  
'  
'  
' Postconditions:  
' 1. Shows the hidden component.  
' 2. Examine the Immediate window, FeatureManager  
'    design tree, and graphics area.  
' -----  
Option Explicit  
  
Sub TraverseComponent(swComp As SldWorks.Component2, nLevel As Long)  
  
    Dim vChildCompArr As Variant  
    Dim vChildComp As Variant  
    Dim swChildComp As SldWorks.Component2  
    Dim swCompConfig As SldWorks.Configuration  
    Dim sPadStr As String  
    Dim i As Long  
  
    For i = 0 To nLevel - 1  
        sPadStr = sPadStr + "  "  
    Next i  
  
    vChildCompArr = swComp.GetChildren  
    For Each vChildComp In vChildCompArr  
        Set swChildComp = vChildComp  
        Debug.Print sPadStr & swChildComp.Name2 & " <" & swChildComp.ReferencedConfiguration & ">"  
        If swComponentHidden = swChildComp.Visible Then  
            swChildComp.Visible = swComponentVisible  
        End If  
        TraverseComponent swChildComp, nLevel + 1  
    Next  
  
End Sub  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swAssy As SldWorks.AssemblyDoc  
    Dim swConf As SldWorks.Configuration  
    Dim swRootComp As SldWorks.Component2  
    Dim bRet As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swConf = swModel.GetActiveConfiguration  
    Set swRootComp = swConf.GetRootComponent  
  
    Debug.Print "File = " & swModel.GetPathName  
  
    TraverseComponent swRootComp, 1  
  
End Sub
```

Make Assembly Components Lightweight Example (VBA)

This example shows how to change the selected components in an assembly from fully resolved to lightweight.

```
' -----
' Preconditions:
' 1. Open an assembly.
' 2. Select the fully resolved components that you want to
'    change to lightweight.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Changes the selected components to lightweight.
' 2. Examine the Immediate window and the FeatureManager
'    design tree.
' -----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swAssy As SldWorks.AssemblyDoc
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swComp As SldWorks.Component2
    Dim nSelCount As Long
    Dim i As Long
    Dim bRet As Boolean
    Dim nRetVal As Long

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swAssy = swModel
    Set swSelMgr = swModel.SelectionManager
    nSelCount = swSelMgr.GetSelectedObjectCount2(-1)
    Debug.Print "Number of selected components = " & nSelCount
    Debug.Print ""

    Debug.Print "Selected components"

    For i = 1 To nSelCount
        Set swComp = swSelMgr.GetSelectedObjectsComponent4(i, -1)
        Debug.Print "  Component = " & swComp.Name2
        Debug.Print "    Path = " & swComp.GetPathName
        Debug.Print "    Suppressed state = " & swComp.GetSuppression
        Debug.Print "    Is suppressed? " & swComp.IsSuppressed
    Next i

    swAssy.MakeLightWeight

    Debug.Print ""
    Debug.Print "Number of lightweight components = " & swAssy.GetLightWeightComponentCount

End Sub
```

Make Assembly from Selected Components Example (C#)

This example shows how to create a new assembly using the selected components of the active assembly.

```
//-----  
// Preconditions:  
// 1. Open public_documents\samples\tutorial\motionstudies\valve_cam2.sldasm  
// 2. Ensure that the Save new components to external files check box  
//    on the Tools > Options > Assemblies dialog is selected.  
//    Otherwise, the selected components are saved as virtual components  
//    and not as external files.  
// 3. Select valve<1> and valve_guide<1> components.  
//  
// Postconditions:  
// 1. Creates public_documents\samples\tutorial\motionstudies\MyTestValveAssembly.sldasm,  
//    which is made up of the valve<1> and valve_guide<1> components.  
// 2. Replaces the valve<1> and valve_guide<1> components with  
//    MyTestValveAssembly subassembly.  
// 3. Examine the FeatureManager design tree and  
//    public_documents\samples\tutorial\motionstudies.  
// 4. Clear the Save new components to external files check box  
//    on the Tools > Options > Assemblies dialog if you selected  
//    it for this example.  
//  
// NOTE: Because the assembly is used elsewhere, do not save changes.  
//-----
```

```
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System;  
using System.Diagnostics;  
namespace MakeAssemblyFromSelectedComponents_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        public void Main()  
        {  
            ModelDoc2 swModel = default(ModelDoc2);  
            AssemblyDoc swAssy = default(AssemblyDoc);  
            string tmpPath = null;  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
            string strCompModelName = null;  
            strCompModelName = "MyTestValveAssembly.sldasm";  
            // Save the new assembly in the same folder as the original assembly  
            tmpPath = swModel.GetPathName();  
            tmpPath = tmpPath.Substring(0, tmpPath.Length - 17);  
            Debug.Print( tmpPath);  
            swAssy = (AssemblyDoc)swModel;  
            // Create a new assembly using the selected components  
            swAssy.MakeAssemblyFromSelectedComponents(tmpPath + strCompModelName);  
        }  
        public SldWorks swApp;  
    }  
}
```

Make Assembly From Selected Components Example (VBA)

This example shows how to create a new assembly using the selected components of the active assembly.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\motionstudies\valve_cam2.sldasm  
' 2. Ensure that the Save new components to external files check box  
'    on the Tools > Options > Assemblies dialog is selected.  
'    Otherwise, the selected components are saved as virtual components  
'    and not as external files.  
' 3. Select valve<1> and valve_guide<1> components.  
'  
' Postconditions:  
' 1. Creates public_documents\samples\tutorial\motionstudies\MyTestValveAssembly.sldasm,  
'    which is made up of the valve<1> and valve_guide<1> components.  
' 2. Replaces the valve<1> and valve_guide<1> components with  
'    MyTestValveAssembly subassembly.  
' 3. Examine the FeatureManager design tree and  
'    public_documents\samples\tutorial\motionstudies.  
' 4. Clear the Save new components to external files check box  
'    on the Tools > Options > Assemblies dialog if you selected  
'    it for this example.  
'  
' NOTE: Because the assembly is used elsewhere, do not save changes.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swAssy As SldWorks.AssemblyDoc  
Dim tmpPath As String  
Dim boolstat As Boolean  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
  
    boolstat = True  
    Dim strCompModelname As String  
    strCompModelname = "MyTestValveAssembly.sldasm"  
  
    ' Save the new assembly in the same folder as the original assembly  
    tmpPath = Left(swModel.GetPathName, InStrRev(swModel.GetPathName, "\"))  
  
    Set swAssy = swModel  
  
    ' Create a new assembly using the selected components  
    swAssy.MakeAssemblyFromSelectedComponents (tmpPath + strCompModelname)
```

End Sub

Make Assembly From Selected Components Example (VB.NET)

This example shows how to make a new assembly using the selected components of the active assembly.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\motionstudies\valve_cam2.sldasm  
' 2. Ensure that the Save new components to external files check box  
'    on the Tools > Options > Assemblies dialog is selected.  
'    Otherwise, the selected components are saved as virtual components  
'    and not as external files.  
' 3. Select valve<1> and valve_guide<1> components.  
'  
' Postconditions:  
' 1. Creates public_documents\samples\tutorial\motionstudies\MyTestValveAssembly.sldasm,  
'    which is made up of the valve<1> and valve_guide<1> components.  
' 2. Replaces the valve<1> and valve_guide<1> components with  
'    MyTestValveAssembly subassembly.  
' 3. Examine the FeatureManager design tree and  
'    public_documents\samples\tutorial\motionstudies.  
' 4. Clear the Save new components to external files check box  
'    on the Tools > Options > Assemblies dialog if you selected  
'    it for this example.  
'  
' NOTE: Because the assembly is used elsewhere, do not save changes.  
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub main()
```

```
        Dim swModel As ModelDoc2
```

```
        Dim swAssy As AssemblyDoc
```

```
        Dim tmpPath As String
```

```
        Dim boolstat As Boolean
```

```
        swModel = swApp.ActiveDoc
```

```
        boolstat = True
```

```
        Dim strCompModelname As String
```

```
        strCompModelname = "MyTestValveAssembly.sldasm"
```

```
        ' Save the new assembly in the same folder as the original assembly
```

```
        tmpPath = Left(swModel.GetPathName, InStrRev(swModel.GetPathName, "\"))
```

```
        swAssy = swModel
```

```
        ' Create a new assembly using the selected components
```

```
swAssy.MakeAssemblyFromSelectedComponents(tmpPath + strCompModelName)
```

```
End Sub
```

```
''' <summary>
```

```
''' The SldWorks swApp variable is pre-assigned for you.
```

```
''' </summary>
```

```
Public swApp As SldWorks
```

```
End Class
```

Make Component Independent Example (C#)

This example shows how to:

- make a selected component independent.
- save the selected component as a new part.
- reference the new part in the assembly.

```
//-----  
// Preconditions:  
// 1. Verify that c:\temp exists.  
// 2. Copy these files from public_documents\samples\tutorial\driveworksexpress  
//    to c:\temp:  
//    * leg.sldprt  
//    * mobile gantry.sldasm  
//    * universal beam.sldpart  
//  
// Postconditions:  
// 1. Opens the assembly.  
// 2. Selects the leg<1> component.  
// 3. Makes the selected component independent, saves the component as  
//    a new part, c:\temp\my leg.sldprt, and references the new part in  
//    the assembly.  
// 4. Examine the FeatureManager design tree and c:\temp.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
  
namespace Macro1CSharp.csproj  
{  
    public partial class SolidWorksMacro  
    {  
  
        public void Main()  
        {  
            ModelDoc2 swModel = default(ModelDoc2);  
            ModelDocExtension swModelDocExt = default(ModelDocExtension);  
            AssemblyDoc swAssembly = default(AssemblyDoc);  
            string fileName = null;  
            bool status = false;  
            int errors = 0;  
            int warnings = 0;  
  
            fileName = "c:\\temp\\mobile gantry.sldasm";  
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);  
            swModelDocExt = (ModelDocExtension)swModel.Extension;  
            swAssembly = (AssemblyDoc)swModel;  
  
            //Select leg<1>  
            status = swModelDocExt.SelectByID2("leg-1@mobile gantry", "COMPONENT", 0, 0, 0, false, 0, null, 0);  
  
            //Make leg<1> independent, save it as a new part,  
            //and reference the new part in the assembly  
            status = swAssembly.MakeIndependent("c:\\temp\\my leg.sldprt");  
  
        }  
  
        /// <summary>  
        /// The SldWorks swApp variable is pre-assigned for you.  
        /// </summary>  
        public SldWorks swApp;  
    }  
}
```

Make Component Independent Example (VBA)

This example shows how to:

- make a selected component independent.
- save the selected component as a new part.
- reference the new part in the assembly.

```
'-----  
' Preconditions:  
' 1. Verify that c:\temp exists.  
' 2. Copy these files from public_documents\samples\tutorial\driveworksexpress  
'    to c:\temp:  
'    * leg.sldprt  
'    * mobile gantry.sldasm  
'    * universal beam.sldpart  
'  
' Postconditions:  
' 1. Opens the assembly.  
' 2. Selects the leg<1> component.  
' 3. Makes the selected component independent, saves the component as  
'    a new part, c:\temp\my leg.sldprt, and references the new part in  
'    the assembly.  
' 4. Examine the FeatureManager design tree and c:\temp.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swAssembly As SldWorks.AssemblyDoc  
Dim fileName As String  
Dim status As Boolean  
Dim errors As Long  
Dim warnings As Long  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    fileName = "c:\temp\mobile gantry.sldasm"  
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY,  
swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)  
    Set swModelDocExt = swModel.Extension  
    Set swAssembly = swModel  
  
    'Select leg<1>  
    status = swModelDocExt.SelectByID2("leg-1@mobile gantry", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)  
  
    'Make leg<1> independent, save it as a new part,  
    'and reference the new part in the assembly  
    status = swAssembly.MakeIndependent("c:\temp\my leg.sldprt")  
  
End Sub
```

Make Component Independent Example (VB.NET)

This example shows how to:

- make a selected component independent.
- save the selected component as a new part.
- reference the new part in the assembly.

```
-----  
' Preconditions:  
' 1. Verify that c:\temp exists.  
' 2. Copy these files from public_documents\samples\tutorial\driveworksexpress  
'    to c:\temp:  
'    * leg.sldprt  
'    * mobile gantry.sldasm  
'    * universal beam.sldpart  
'  
' Postconditions:  
' 1. Opens the assembly.  
' 2. Selects the leg<1> component.  
' 3. Makes the selected component independent, saves the component as  
'    a new part, c:\temp\my leg.sldprt, and references the new part in  
'    the assembly.  
' 4. Examine the FeatureManager design tree and c:\temp.  
-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System
```

Partial Class SolidWorksMacro

```
    Public Sub main()  
        Dim swModel As ModelDoc2  
        Dim swModelDocExt As ModelDocExtension  
        Dim swAssembly As AssemblyDoc  
        Dim fileName As String  
        Dim status As Boolean  
        Dim errors As Integer  
        Dim warnings As Integer  
  
        fileName = "c:\temp\mobile gantry.sldasm"  
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)  
        swModelDocExt = swModel.Extension  
        swAssembly = swModel  
  
        'Select leg<1>  
        status = swModelDocExt.SelectByID2("leg-1@mobile gantry", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)  
  
        'Make leg<1> independent, save it as a new part,  
        'and reference the new part in the assembly  
        status = swAssembly.MakeIndependent("c:\temp\my leg.sldprt")  
  
    End Sub  
  
    ''' <summary>  
    ''' The SldWorks swApp variable is pre-assigned for you.  
    ''' </summary>  
    Public swApp As SldWorks
```

End Class

Make Line Infinite Example (VBA)

This example shows how to make the selected finite line an infinite line.

```
'-----
' Preconditions:
' 1. Verify that the specified part template exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens a new part.
' 2. Sketches a finite line and sets its angle.
' 3. Changes the finite line to an infinite line and
'    changes the angle of the infinite line.
' 4. Examine the graphics area and Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchMgr As SldWorks.SketchManager
Dim swSketchLine As SldWorks.SketchLine
Dim boolstatus As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2016\templates\Part.prtdot", 0, 0, 0)
    Set swModelDocExt = swModel.Extension

    'Sketch a finite line and change its angle
    boolstatus = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    Set swSketchMgr = swModel.SketchManager
    swSketchMgr.InsertSketch True
    Set swSketchLine = swSketchMgr.CreateLine(-0.064203, -0.00825, 0#, -0.009545, 0.029422, 0#)
    swSketchLine.Angle = 0.785
    swModel.ClearSelection2 True

    boolstatus = swSketchLine.Infinite
    Debug.Print "Infinite line? " & boolstatus
    Debug.Print "Angle? " & swSketchLine.Angle

    'Change finite line to infinite line
    'and change its angle
    boolstatus = swSketchLine.MakeInfinite
    boolstatus = swSketchLine.Infinite
    Debug.Print "Infinite line? " & boolstatus
    swSketchLine.Angle = 1#
    Debug.Print "Angle? " & swSketchLine.Angle

    swSketchMgr.InsertSketch True

End Sub
```

Make Part Transparent Example (VBA)

This example shows how to make a part transparent.

```
'-----  
' Preconditions:  
' 1. Open a part.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Makes the part transparent.  
' 2. Examine the graphics area and Immediate window.  
'-----
```

Option Explicit

```
Function GetRValue(MyColour As Long) As Long  
    GetRValue = MyColour Mod 256  
End Function
```

```
Function GetGValue(MyColour As Long) As Long  
    GetGValue = (MyColour \ 256) Mod 256  
End Function
```

```
Function GetBValue(MyColour As Long) As Long  
    GetBValue = (MyColour \ 65536) Mod 256  
End Function
```

Sub main()

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swPart As SldWorks.PartDoc  
Dim nDefaultColor As Long  
Dim vBodyArr As Variant  
Dim vBody As Variant  
Dim swBody As SldWorks.Body2  
Dim swFace As SldWorks.Face2  
Dim nMatProp(9) As Double  
Dim vMatProp As Variant  
Dim bRet As Boolean
```

```
Set swApp = Application.SldWorks  
Set swModel = swApp.ActiveDoc  
Set swPart = swModel
```

```
nDefaultColor = swModel.GetUserPreferenceIntegerValue(swDocumentColorShading)
```

```
Debug.Print "File = " & swModel.GetPathName
```

```
Debug.Print " Default Color = RGB(" & GetRValue(nDefaultColor) & ", " & GetGValue(nDefaultColor) & ", " &  
GetBValue(nDefaultColor) & ")"
```

```
' Set to sensible defaults  
' Gray for default color  
nMatProp(0) = GetRValue(nDefaultColor) / 255#           ' red  
nMatProp(1) = GetRValue(nDefaultColor) / 255#           ' green  
nMatProp(2) = GetRValue(nDefaultColor) / 255#           ' blue  
nMatProp(3) = 1#                                         ' Ambient  
nMatProp(4) = 1#                                         ' Diffuse  
nMatProp(5) = 1#                                         ' Specular  
nMatProp(6) = 0.31                                       ' Shininess  
' Increase transparency  
nMatProp(7) = 0.95                                       ' Transparency  
nMatProp(8) = 0#                                         ' Emission  
vMatProp = nMatProp
```

```
vBodyArr = swPart.GetBodies2(swAllBodies, True)
```

```
For Each vBody In vBodyArr  
    Set swBody = vBody  
    Set swFace = swBody.GetFirstFace
```

```
While Not swFace Is Nothing
    swFace.MaterialPropertyValues = vMatProp
    Set swFace = swFace.GetNextFace
Wend
Next

' Redraw to see new transparency
swModel.GraphicsRedraw2

End Sub
```

Make Smart Component Example (VBA)

This example shows how to create a Smart Component.

```
' -----  
' Preconditions:  
' 1. Open an assembly containing three components.  
' 2. Change names of the selected components in  
'    the macro to match the names of the components  
'    in your assembly.  
'  
' Postconditions:  
' 1. Creates a Smart Component.  
' 2. Expand the first selected component and  
'    expand and examine the Smart Feature folder.  
' -----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swAssembly As SldWorks.AssemblyDoc  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim pCompSmart As Object  
Dim pComp1 As Object  
Dim pComp2 As Object  
Dim pCompArr(1) As Object  
Dim relcomp As Variant  
Dim relfeat As Variant  
Dim boundval As Variant  
Dim boolstatus As Boolean  
Dim retval As Boolean
```

Sub main()

```
Set swApp = Application.SldWorks  
Set swModel = swApp.ActiveDoc  
Set swAssembly = swModel  
Set swSelMgr = swModel.SelectionManager  
Set swModelDocExt = swModel.Extension  
  
' Select the component that you want to make smart  
boolstatus = swModelDocExt.SelectByID2("C-1@ABC_assembly", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)  
  
' Select the components that you want to associate with the Smart Component  
boolstatus = swModelDocExt.SelectByID2("B-1@ABC_assembly", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)  
boolstatus = swModelDocExt.SelectByID2("A-2@ABC_assembly", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)  
  
Set pCompSmart = swSelMgr.GetSelectedObject6(1, 0)  
Set pComp1 = swSelMgr.GetSelectedObject6(2, 0)  
Set pComp2 = swSelMgr.GetSelectedObject6(3, 0)  
  
Set pCompArr(0) = pComp1  
Set pCompArr(1) = pComp2  
  
relcomp = pCompArr  
  
swModel.ClearSelection2 True  
  
' Create the Smart Component  
retval = swAssembly.CreateSmartComponent(pCompSmart, (relcomp), (relfeat), False, Nothing, boundval)  
  
End Sub
```

Make Smart Component with Mate Example (VBA)

This example shows how to create a Smart Component with a concentric mate reference.

```
'-----  
' Preconditions:  
' 1. Verify that the specified part and assembly templates exist.  
' 2. Verify that C:\temp exists.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Creates a new part.  
' 2. Creates a new assembly using the part as a component.  
' 3. Adds two more instances of the part as components.  
' 4. Creates a Smart Component with a concentric mate reference.  
' 5. Examine the Immediate window and FeatureManager design tree.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSketchSegment As SldWorks.SketchSegment  
Dim swSketchMgr As SldWorks.SketchManager  
Dim swFeature As SldWorks.Feature  
Dim swFeatureMgr As SldWorks.FeatureManager  
Dim swSelectionMgr As SldWorks.SelectionMgr  
Dim swAssembly As SldWorks.AssemblyDoc  
Dim status As Boolean  
Dim errors As Long  
Dim warnings As Long  
Dim pMateRef As Object  
Dim pCompSmart As Object  
Dim pComp1 As Object  
Dim pComp2 As Object  
Dim pFeatArr(0) As Object  
Dim pCompArr(1) As Object  
Dim relcomp As Variant  
Dim relfeat As Variant  
Dim boundval As Variant  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
  
    'Create part  
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2016\templates\Part.prtdot", 0, 0, 0)  
    Set swModelDocExt = swModel.Extension  
    status = swModelDocExt.SelectByID("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)  
    swModel.ActivateSelectedFeature  
    swModel.ClearSelection2 True  
    Set swSketchMgr = swModel.SketchManager  
    swSketchMgr.InsertSketch True  
    Set swSketchSegment = swSketchMgr.CreateLine(0#, 0#, 0#, 0#, -0.033793, 0#)  
    status = swModelDocExt.SelectByID("Line1", "SKETCHSEGMENT", 0, 0, 0, False, 16, Nothing, 0)  
    Set swFeatureMgr = swModel.FeatureManager  
    Set swFeature = swFeatureMgr.FeatureRevolve2(True, True, True, False, False, False, 0, 0, 6.2831853071796, 0,  
False, False, 0.01, 0.01, 0, 0.01, 0.01, True, True, True)  
    status = swModelDocExt.SaveAs("C:\temp\RevolveComponent.SLDPRT", swSaveAsVersion_e.swSaveAsCurrentVersion,  
swSaveAsOptions_e.swSaveAsOptions_Silent, Nothing, errors, warnings)  
  
    'Create assembly  
    Set swAssembly = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2016\templates\Assembly.asmdot", 0,  
0, 0)  
    status = swAssembly.AddComponent("C:\temp\RevolveComponent.SLDPRT", 6.25313657656079E-03, 1.63811589345642E-  
04, -3.41098612290125E-03)  
    status = swAssembly.AddComponent("C:\temp\RevolveComponent.SLDPRT", -2.72449827753007E-03, 3.30138755962253E-  
02, -3.17469704328105E-02)  
    status = swAssembly.AddComponent("C:\temp\RevolveComponent.SLDPRT", 0.018507689004764, -4.20309320325032E-02,
```

```

1.74126345664263E-02)
Set swModel = swAssembly
Set swModelDocExt = swModel.Extension
status = swModelDocExt.SaveAs("C:\temp\RevolveComponentsAssembly.sldasm",
swSaveAsVersion_e.swSaveAsCurrentVersion, swSaveAsOptions_e.swSaveAsOptions_Silent, Nothing, errors, warnings)

' Select component to make smart
status = swModelDocExt.SelectByID2("RevolveComponent-1@RevolveComponentsAssembly", "COMPONENT", 0, 0, 0,
False, 0, Nothing, 0)

' Select components to associate with Smart Component
status = swModelDocExt.SelectByID2("RevolveComponent-2@RevolveComponentsAssembly", "COMPONENT", 0, 0, 0,
True, 0, Nothing, 0)
status = swModelDocExt.SelectByID2("RevolveComponent-3@RevolveComponentsAssembly", "COMPONENT", 0, 0, 0,
True, 0, Nothing, 0)

' Select an face on a component for the concentric mate reference
status = swModelDocExt.SelectByID2("", "FACE", 7.12645683466917E-03, 5.00071834983373E-03, 6.55080647294426E-
03, True, 0, Nothing, 0)

Set swSelectionMgr = swModel.SelectionManager
Set pCompSmart = swSelectionMgr.GetSelectedObject6(1, 0)
Set pComp1 = swSelectionMgr.GetSelectedObject6(2, 0)
Set pComp2 = swSelectionMgr.GetSelectedObject6(3, 0)
Set pMateRef = swSelectionMgr.GetSelectedObject6(4, 0)

Set pCompArr(0) = pComp1
Set pCompArr(1) = pComp2

relcomp = pCompArr

swModel.ClearSelection2 True

'Create Smart Component with concentric mate reference
status = swAssembly.CreateSmartComponent(pCompSmart, (relcomp), (relfeat), True, pMateRef, boundval)
Debug.Print "Smart component created? " & status

End Sub

```

Manage Collision Detection Example (C#)

This example shows how to manage collision detection.

```
//-----  
// Preconditions:  
// 1. Open public_documents\samples\tutorial\api\Spindleassem.sldasm.  
// 2. Open the Immediate window.  
//  
// Postconditions: Inspect the Immediate window.  
//  
// NOTE: Because the model is used elsewhere, do not save changes to it.  
//-----  
  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
using System.Diagnostics;  
  
namespace ManageCollisionDetection  
{  
    partial class SolidWorksMacro  
    {  
  
        private ModelDoc2 Part;  
        private AssemblyDoc swDoc;  
        private CollisionDetectionManager cdm;  
        private CollisionDetectionGroup cdg;  
        private CollisionDetectionGroup cdg2;  
        private CollisionDetectionGroup cdg3;  
        private Collision aCollision;  
        private double[] TransformData = new double[16];  
        private object TransformDataVariant;  
        private double[] TransformData1 = new double[16];  
        private object TransformDataVariant1;  
        private Component2[] comp = new Component2[1];  
        private Component2[] comp1 = new Component2[1];  
        private Component2[] comp2 = new Component2[1];  
        private MathTransform[] transform = new MathTransform[1];  
        private MathTransform[] transform1 = new MathTransform[1];  
        private MathUtility swMathUtil;  
        private object var1;  
        private bool boolstatus;  
        private int longstatus;  
        private int numCollisions;  
        private int i;  
        private int j;  
        private object[] comps;  
  
        public void Main()  
        {  
  
            Part = (ModelDoc2)swApp.ActiveDoc;  
            swDoc = (AssemblyDoc)Part;  
            cdm = (CollisionDetectionManager)swApp.GetCollisionDetectionManager();  
            longstatus = cdm.SetAssembly(swDoc);  
  
            cdm.GraphicsRedrawEnabled = true;  
  
            cdg = (CollisionDetectionGroup)cdm.CreateGroup();
```

```

cdg2 = (CollisionDetectionGroup)cdm.CreateGroup();
cdg3 = (CollisionDetectionGroup)cdm.CreateGroup();

//Holder
boolstatus = Part.Extension.SelectByID2("Holder-
1@Spindleassem", "COMPONENT", 0, 0, 0, true, 0, null, 0);

TransformData[0] = 0.561729092855705;
TransformData[1] = -0.827321235216108;
TransformData[2] = 0;
TransformData[3] = 0.827321235216108;
TransformData[4] = 0.561729092855705;
TransformData[5] = 0;
TransformData[6] = 0;
TransformData[7] = 0;
TransformData[8] = 1.00;
TransformData[9] = 0.0202097529022064;
TransformData[10] = 0.025899850979138;
TransformData[11] = 0.1;
TransformData[12] = 1;
TransformData[13] = 0;
TransformData[14] = 0;
TransformData[15] = 0;

TransformDataVariant = TransformData;
swMathUtil = (MathUtility)swApp.GetMathUtility();

//Cutting tool
boolstatus = Part.Extension.SelectByID2("CuttingTool-
1@Spindleassem", "COMPONENT", 0, 0, 0, true, 0, null, 0);

TransformData1[0] = 1;
TransformData1[1] = 0;
TransformData1[2] = 0;
TransformData1[3] = 0;
TransformData1[4] = 1;
TransformData1[5] = 0;
TransformData1[6] = 0;
TransformData1[7] = 0;
TransformData1[8] = 1;
TransformData1[9] = -0.01;
TransformData1[10] = -0.03;
TransformData1[11] = 0.1;
TransformData1[12] = 1;
TransformData1[13] = 0;
TransformData1[14] = 0;
TransformData1[15] = 0;

TransformDataVariant1 = TransformData1;

//Work piece
boolstatus = Part.Extension.SelectByID2("workpiece-
1@Spindleassem", "COMPONENT", 0, 0, 0, true, 0, null, 0);

comp[0] = (Component2)((SelectionMgr)
(Part.SelectionManager)).GetSelectedObjectsComponent4(1, -1);
comp1[0] = (Component2)((SelectionMgr)
(Part.SelectionManager)).GetSelectedObjectsComponent4(2, -1);
comp2[0] = (Component2)((SelectionMgr)
(Part.SelectionManager)).GetSelectedObjectsComponent4(3, -1);

transform[0] = (MathTransform)swMathUtil.CreateTransform((TransformDataVariant));
transform1[0] = (MathTransform)swMathUtil.CreateTransform((TransformDataVariant1));

longstatus = cdg.SetComponents(comp); //Holder
longstatus = cdg2.SetComponents(comp1); //Cutting tool
longstatus = cdg3.SetComponents(comp2); //Work piece

longstatus = cdg.ApplyTransforms(transform);
longstatus = cdg2.ApplyTransforms(transform1);

```

```

longstatus = cdm.IsCollisionPresent(true);
numCollisions = cdm.GetCollisions(true, out var1);

object[] collisions;
collisions = (object[])var1;

if (numCollisions > 0)
{
    int tempVar = numCollisions - 1;
    for (i = 0; i <= tempVar; i++)
    {
        Debug.Print("Collision " + (i + 1));
        aCollision = (Collision)collisions[i];
        Debug.Print("  Is penetrating? " + aCollision.IsPenetrating());
        comps = (object[])aCollision.GetComponents();
        for (j = 0; j <= comps.GetUpperBound(0); j++)
        {
            Debug.Print("    " + ((Component2)comps[j]).Name);
        }
    }
}
else
{
    Debug.Print("No collisions present");
}

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Manage Drawing Document Line Styles Example (C#)

This example shows how to manage the line styles of a drawing document.

```
//-----  
// Preconditions:  
// 1. Verify that the specified drawing document template exists.  
// 2. Ensure that c:\temp exists.  
// 3. Open an Immediate Window.  
//  
// Postconditions:  
// 1. Opens a new drawing.  
// 2. Adds line styles.  
// 3. Saves line styles.  
// 4. Deletes a line style.  
// 5. Loads saved line styles, replacing existing line styles.  
// 6. Inspect the Immediate window.  
//  
// NOTE: Line styles are saved to c:\temp\styles.sldlin.  
//-----  
  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System;  
using System.Diagnostics;  
namespace DrawingLineStyles_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        DrawingDoc Part;  
        bool boolstatus;  
        int longstatus;  
  
        public void Main()  
        {  
            string def = null;  
            string name = null;  
  
            Part = (DrawingDoc)swApp.NewDocument("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS  
2016\\templates\\drawing.drwdot", 2, 0.2794, 0.4318);  
            swApp.ActivateDoc2("Draw1 - Sheet1", false, ref longstatus);  
  
            printData("Line Style Data at Start", "");  
  
            def = "B,1.2,-0.2,2,-0.1,2";  
            name = "NewOne";  
            boolstatus = Part.AddLineStyle(name, def);  
            printData("Line Style Data After Add", "");  
  
            object names = null;  
            string[] styleNames = new string[3];  
  
            styleNames[0] = "NewOne";  
            styleNames[1] = "CHAIN";  
            styleNames[2] = "PHANTOM";  
  
            names = styleNames;  
  
            // Save line styles  
            boolstatus = Part.SaveLineStyles("c:\\temp\\styles", names, true);  
            printData("Line Style Data saved to file ", "c:\\temp\\styles");  
  
            // Delete a line style  
            boolstatus = Part.DeleteLineStyle("NewOne", "STITCH");  
        }  
    }  
}
```

```

        printData("Line Style Data After Delete", "");

        // Load saved line styles, replacing existing line styles
        bool status = Part.LoadLineStyles("c:\\temp\\styles", names, true);
        printData("Line Style Data Imported from file", "");
    }

    public void printData(string title, string file)
    {
        object names = null;
        object types = null;
        string[] namesArr = null;
        string[] typesArr = null;
        int i = 0;
        bool stat = false;

        Debug.Print("-----");
        Debug.Print(title);
        Debug.Print("-----");

        if (string.IsNullOrEmpty(file))
        {
            stat = Part.GetLineStyles(out names, out types);
        }
        else
        {
            stat = swApp.GetLineStyles(file, out names, out types);
        }
        namesArr = (string[])names;
        typesArr = (string[])types;
        if (stat)
        {
            for (i = 0; i <= typesArr.GetUpperBound(0); i++)
            {
                Debug.Print(i + " " + namesArr[i] + " " + typesArr[i]);
            }
        }
        else
        {
            Debug.Print("Error in printData");
        }
    }
}
public SldWorks swApp;
}
}

```

Manage Drawing Document Line Styles Example (VBA)

This example shows how to manage the line styles of a drawing document.

```
'-----  
' Preconditions:  
' 1. Verify that the specified drawing document template exists.  
' 2. Ensure that c:\temp exists.  
' 3. Open an Immediate Window.  
'  
' Postconditions:  
' 1. Opens a new drawing.  
' 2. Adds line styles.  
' 3. Saves line styles.  
' 4. Deletes a line style.  
' 5. Loads saved line styles, replacing existing line styles.  
' 6. Inspect the Immediate window.  
'  
' NOTE: Line styles are saved to c:\temp\styles.sldlin.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
```

```
Dim Part As DrawingDoc
```

```
Dim boolstatus As Boolean
```

```
Dim longstatus As Long
```

```
Sub main()
```

```
    Dim def As String
```

```
    Dim name As String
```

```
    Set swApp = Application.SldWorks
```

```
    Set Part = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS  
2016\templates\drawing.drwdot", 2, 0.2794, 0.4318)
```

```
    swApp.ActivateDoc2 "Draw1 - Sheet1", False, longstatus
```

```
    printData "Line Style Data at Start", ""
```

```
    def = "B,1.2,-0.2,2,-0.1,2"
```

```
    name = "NewOne"
```

```
    boolstatus = Part.AddLineStyle(name, def)
```

```
    printData "Line Style Data After Add", ""
```

```
    Dim names As Variant
```

```
    Dim styleNames(2) As String
```

```
    styleNames(0) = "NewOne"
```

```
    styleNames(1) = "CHAIN"
```

```
    styleNames(2) = "PHANTOM"
```

```
    names = styleNames
```

```
    ' Save line styles
```

```
    boolstatus = Part.SaveLineStyles("c:\temp\styles", names, True)
```

```
    printData "Line Style Data saved to file ", "c:\temp\styles"
```

```
    ' Delete a line style
```

```
    boolstatus = Part.DeleteLineStyle("NewOne", "STITCH")
```

```

printData "Line Style Data After Delete", ""

' Load saved line styles, replacing existing line styles
boolstatus = Part.LoadLineStyles("c:\temp\styles", names, True)
printData "Line Style Data Imported from file", ""
End Sub

Sub printData(title As String, file As String)
    Dim names As Variant
    Dim types As Variant
    Dim i As Integer
    Dim stat As Boolean

    Debug.Print Chr$(10) + "-----"
    Debug.Print title
    Debug.Print "-----"

    If file = "" Then
        stat = Part.GetLineStyles(names, types)
    Else
        stat = swApp.GetLineStyles(file, names, types)
    End If

    If stat Then
        For i = 0 To UBound(types)
            Debug.Print Str$(i) + " ", names(i) + " " + types(i)
        Next i
    Else
        MsgBox "Error in printData"
    End If
End Sub

```

Manage Drawing Document Line Styles Example (VB.NET)

This example shows how to manage the line styles of a drawing document.

```
'-----  
' Preconditions:  
' 1. Verify that the specified drawing document template exists.  
' 2. Ensure that c:\temp exists.  
' 3. Open an Immediate Window.  
'  
' Postconditions:  
' 1. Opens a new drawing.  
' 2. Adds line styles.  
' 3. Saves lines styles.  
' 4. Deletes a line style.  
' 5. Loads saved line styles, replacing existing line styles.  
' 6. Inspect the Immediate window.  
'  
' NOTE: Line styles are saved to c:\temp\styles.sldlin.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System  
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Dim Part As DrawingDoc  
    Dim boolstatus As Boolean  
    Dim longstatus As Long
```

```
    Sub main()
```

```
        Dim def As String  
        Dim name As String
```

```
        Part = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS  
2016\templates\drawing.drwdot", 2, 0.2794, 0.4318)  
        swApp.ActivateDoc2("Draw1 - Sheet1", False, longstatus)
```

```
        printData("Line Style Data at Start", "")
```

```
        def = "B,1.2,-0.2,2,-0.1,2"  
        name = "NewOne"  
        boolstatus = Part.AddLineStyle(name, def)  
        printData("Line Style Data After Add", "")
```

```
        Dim names As Object  
        Dim styleNames(2) As String
```

```
        styleNames(0) = "NewOne"  
        styleNames(1) = "CHAIN"  
        styleNames(2) = "PHANTOM"
```

```
        names = styleNames
```

```
        ' Save line styles  
        boolstatus = Part.SaveLineStyles("c:\temp\styles", names, True)  
        printData("Line Style Data saved to file ", "c:\temp\styles")
```

```
        ' Delete a line style  
        boolstatus = Part.DeleteLineStyle("NewOne", "STITCH")  
        printData("Line Style Data After Delete", "")
```

```

' Load saved line styles, replacing existing line styles
boolstatus = Part.LoadLineStyles("c:\temp\styles", names, True)
printData("Line Style Data Imported from file", "")

```

End Sub

```

Sub printData(ByVal title As String, ByVal file As String)

```

```

    Dim names As Object = Nothing
    Dim types As Object = Nothing
    Dim namesArr() As String
    Dim typesArr() As String
    Dim i As Integer
    Dim stat As Boolean

```

```

    Debug.Print("-----")
    Debug.Print(title)
    Debug.Print("-----")

```

```

    If file = "" Then
        stat = Part.GetLineStyles(names, types)
    Else
        stat = swApp.GetLineStyles(file, names, types)
    End If

```

```

    namesArr = names
    typesArr = types

```

```

    If stat Then

```

```

        For i = 0 To UBound(typesArr)

```

```

            Debug.Print(Str$(i) + " " + namesArr(i) + " " + typesArr(i))

```

```

        Next i

```

```

    Else
        MsgBox("Error in printData")
    End If

```

End Sub

```

Public swApp As SldWorks

```

End Class

Measure Selected Entities Example (C#)

This example shows how to use the measure tool.

```
//-----  
// Preconditions:  
// 1. Verify that the specified part document exists.  
// 2. Open the Immediate window.  
//  
// Postconditions:  
// 1. Opens the specified part document as read only.  
// 2. Selects two faces and measures them.  
// 3. Examine the Immediate window.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
using System.Diagnostics;  
  
namespace MeasureCSharp.csproj  
{  
    public partial class SolidWorksMacro  
    {  
        public void Main()  
        {  
            ModelDoc2 swModel = default(ModelDoc2);  
            ModelDocExtension swModelDocExt = default(ModelDocExtension);  
            Measure swMeasure = default(Measure);  
            bool status = false;  
            int errors = 0;  
            int warnings = 0;  
            string fileName = null;  
  
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS  
2018\\samples\\tutorial\\dimxpert\\coupling_auto_geo.sldprt";  
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_  
e.swOpenDocOptions_ReadOnly, "", ref errors, ref warnings);  
            swModelDocExt = (ModelDocExtension)swModel.Extension;  
            status = swModelDocExt.SelectByID2("", "FACE", -0.00382117299216134, -0.0032246917626253, -0.001538540  
43344381, false, 0, null, 0);  
            status = swModelDocExt.SelectByID2("", "FACE", 0.00547600669648318, 0.00252191841298099, 0.00500000000  
00523, true, 0, null, 0);  
            swMeasure = (Measure)swModelDocExt.CreateMeasure();  
            //Can set this to 0  
            // 0 = center to center  
            // 1 = minimum distance  
            // 2 = maximum distance  
            swMeasure.ArcOption = 0;  
  
            status = swMeasure.Calculate(null);  
            if ((status))  
            {  
                if ((!(swMeasure.Length == -1)))  
                {  
                    Debug.Print("Length: " + swMeasure.Length);  
                }  
                if ((!(swMeasure.Area == -1)))  
                {  
                    Debug.Print("Area: " + swMeasure.Area);  
                }  
                if ((!(swMeasure.ArcLength == -1)))  
                {  
                    Debug.Print("Arc length: " + swMeasure.ArcLength);  
                }  
                if ((!(swMeasure.ChordLength == -1)))  
                {  
                    Debug.Print("Chord length: " + swMeasure.ChordLength);  
                }  
                if ((!(swMeasure.Diameter == -1)))  
                {  
                    Debug.Print("Diameter: " + swMeasure.Diameter);  
                }  
                if ((!(swMeasure.Radius == -1)))  
                {  
                    Debug.Print("Radius: " + swMeasure.Radius);  
                }  
                if ((!(swMeasure.Perimeter == -1)))  
            }  
        }  
    }  
}
```

```

    {
        Debug.Print("Perimeter: " + swMeasure.Perimeter);
    }
    if ((!(swMeasure.X == -1)))
    {
        Debug.Print("X coordinate: " + swMeasure.X);
    }
    if ((!(swMeasure.Y == -1)))
    {
        Debug.Print("Y coordinate: " + swMeasure.Y);
    }
    if ((!(swMeasure.Z == -1)))
    {
        Debug.Print("Z coordinate: " + swMeasure.Z);
    }
    if ((!(swMeasure.DeltaX == -1)))
    {
        Debug.Print("DeltaX: " + swMeasure.DeltaX);
    }
    if ((!(swMeasure.DeltaY == -1)))
    {
        Debug.Print("DeltaY: " + swMeasure.DeltaY);
    }
    if ((!(swMeasure.DeltaZ == -1)))
    {
        Debug.Print("DeltaZ: " + swMeasure.DeltaZ);
    }
    if ((!(swMeasure.Angle == -1)))
    {
        Debug.Print("Angle: " + swMeasure.Angle);
    }
    if ((!(swMeasure.CenterDistance == -1)))
    {
        Debug.Print("Center distance: " + swMeasure.CenterDistance);
    }
    if ((!(swMeasure.NormalDistance == -1)))
    {
        Debug.Print("Normal distance: " + swMeasure.NormalDistance);
    }
    if ((!(swMeasure.Distance == -1)))
    {
        Debug.Print("Distance: " + swMeasure.Distance);
    }
    if ((!(swMeasure.TotalLength == -1)))
    {
        Debug.Print("Total length: " + swMeasure.TotalLength);
    }
    if ((!(swMeasure.TotalArea == -1)))
    {
        Debug.Print("Total area: " + swMeasure.TotalArea);
    }
    if ((swMeasure.IsParallel))
    {
        Debug.Print("Is parallel: " + swMeasure.IsParallel);
    }
    if ((swMeasure.IsIntersect))
    {
        Debug.Print("Is intersect: " + swMeasure.IsIntersect);
    }
    if ((swMeasure.IsPerpendicular))
    {
        Debug.Print("Is perpendicular: " + swMeasure.IsPerpendicular);
    }
    if ((!(swMeasure.Projection == -1)))
    {
        Debug.Print("Projection: " + swMeasure.Projection);
    }
    if ((!(swMeasure.Normal == -1)))
    {
        Debug.Print("Normal: " + swMeasure.Normal);
    }
    if ((!(swMeasure.SphericalCenterDistance == -1)))
    {
        Debug.Print("Spherical center distance: " + swMeasure.SphericalCenterDistance);
    }
    if ((swMeasure.IsConcentricSpheres))
    {
        Debug.Print("Are concentric spheres: " + swMeasure.IsConcentricSpheres);
    }
}
else
{
    Debug.Print("Invalid combination of selected entities.");
}
swModel.ClearSelection2(true);
}

/// <summary>

```

```
    /// The SldWorks swApp variable is pre-assigned for you.  
    /// </summary>  
    public SldWorks swApp;  
}
```

Measure Selected Entities Example (VBA)

This example shows how to use the measure tool.

```
'-----  
' Preconditions:  
' 1. Verify that the specified part document exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens the specified part document as read only.  
' 2. Selects two faces and measures them.  
' 3. Examine the Immediate window.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swMeasure As SldWorks.Measure  
Dim status As Boolean  
Dim errors As Long  
Dim warnings As Long  
Dim fileName As String  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS  
2018\samples\tutorial\dimxpert\coupling_auto_geo.sldprt"  
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART,  
swOpenDocOptions_e.swOpenDocOptions_ReadOnly, "", errors, warnings)  
    Set swModelDocExt = swModel.Extension  
    status = swModelDocExt.SelectByID("", "FACE", -3.82117299216134E-03, -3.2246917626253E-03,  
-1.53854043344381E-03, False, 0, Nothing, 0)  
    status = swModelDocExt.SelectByID("", "FACE", 5.47600669648318E-03, 2.52191841298099E-03, 5.0000000000523E-  
03, True, 0, Nothing, 0)  
  
    Set swMeasure = swModelDocExt.CreateMeasure  
    'Can set this to 0  
    ' 0 = center to center  
    ' 1 = minimum distance  
    ' 2 = maximum distance  
    swMeasure.ArcOption = 0  
  
    status = swMeasure.Calculate(Nothing)  
    If (status) Then  
  
        If (Not (swMeasure.Length = -1)) Then  
            Debug.Print "Length: " & swMeasure.Length  
        End If  
  
        If (Not (swMeasure.Area = -1)) Then  
            Debug.Print "Area: " & swMeasure.Area  
        End If  
  
        If (Not (swMeasure.ArcLength = -1)) Then  
            Debug.Print "Arc length: " & swMeasure.ArcLength  
        End If  
  
        If (Not (swMeasure.ChordLength = -1)) Then  
            Debug.Print "Chord length: " & swMeasure.ChordLength  
        End If  
  
        If (Not (swMeasure.Diameter = -1)) Then  
            Debug.Print "Diameter: " & swMeasure.Diameter  
        End If  
    End If  
End Sub
```

```

If (Not (swMeasure.Radius = -1)) Then
    Debug.Print "Radius: " & swMeasure.Radius
End If

If (Not (swMeasure.Perimeter = -1)) Then
    Debug.Print "Perimeter: " & swMeasure.Perimeter
End If

If (Not (swMeasure.X = -1)) Then
    Debug.Print "X coordinate: " & swMeasure.X
End If

If (Not (swMeasure.Y = -1)) Then
    Debug.Print "Y coordinate: " & swMeasure.Y
End If

If (Not (swMeasure.Z = -1)) Then
    Debug.Print "Z coordinate: " & swMeasure.Z
End If

If (Not (swMeasure.DeltaX = -1)) Then
    Debug.Print "DeltaX: " & swMeasure.DeltaX
End If

If (Not (swMeasure.DeltaY = -1)) Then
    Debug.Print "DeltaY: " & swMeasure.DeltaY
End If

If (Not (swMeasure.DeltaZ = -1)) Then
    Debug.Print "DeltaZ: " & swMeasure.DeltaZ
End If

If (Not (swMeasure.Angle = -1)) Then
    Debug.Print "Angle: " & swMeasure.Angle
End If

If (Not (swMeasure.CenterDistance = -1)) Then
    Debug.Print "Center distance: " & swMeasure.CenterDistance
End If

If (Not (swMeasure.NormalDistance = -1)) Then
    Debug.Print "Normal distance: " & swMeasure.NormalDistance
End If

If (Not (swMeasure.Distance = -1)) Then
    Debug.Print "Distance: " & swMeasure.Distance
End If

If (Not (swMeasure.TotalLength = -1)) Then
    Debug.Print "Total length: " & swMeasure.TotalLength
End If

If (Not (swMeasure.TotalArea = -1)) Then
    Debug.Print "Total area: " & swMeasure.TotalArea
End If

If (swMeasure.IsParallel) Then
    Debug.Print "Is parallel: " & swMeasure.IsParallel
End If

If (swMeasure.IsIntersect) Then
    Debug.Print "Is intersect: " & swMeasure.IsIntersect
End If

If (swMeasure.IsPerpendicular) Then
    Debug.Print "Is perpendicular: " & swMeasure.IsPerpendicular
End If

If (Not (swMeasure.Projection = -1)) Then
    Debug.Print "Projection: " & swMeasure.Projection
End If

If (Not (swMeasure.Normal = -1)) Then
    Debug.Print "Normal: " & swMeasure.Normal
End If

If (Not (swMeasure.SphericalCenterDistance = -1)) Then
    Debug.Print "Spherical center distance: " & swMeasure.SphericalCenterDistance
End If

```

```
    If (swMeasure.IsConcentricSpheres) Then
        Debug.Print "Are concentric spheres: " & swMeasure.IsConcentricSpheres
    End If

Else

    Debug.Print "Invalid combination of selected entities."

End If

swModel.ClearSelection2 True

End Sub
```